

BDA Course 2026

Lecture 5

Chapter 11

- 11.1 Gibbs sampler
- 11.2 Metropolis and Metropolis–Hastings
- 11.3 Using Gibbs and Metropolis as building blocks
- 11.4 Inference and assessing convergence (important)
 - potential scale reduction $\hat{R}$ (R-hat)
- 11.5 Effective number of simulation draws (important)
 - effective sample size (ESS / S_{eff})
- 11.6 Example: hierarchical normal model (quick glance)

Objectives of this Lecture

- Discuss the core ideas behind MCMC methods, a family of very general sampling algorithms that are the backbone of modern Bayesian analysis
- Discuss practical diagnostics we use to check these MCMC methods

Monte Carlo Methods (Reminder)

- Monte Carlo methods are used to approximate expectations by averaging over random samples
- If we can draw samples $\theta^1, \dots, \theta^S$ from a distribution $p(\theta)$, then for any function $h(\theta)$, define the Monte Carlo estimate

$$\mathbb{E}[h(\theta)] \approx \hat{h}_S = \frac{1}{S} \sum_{s=1}^S h(\theta^s)$$

- By CLT

$$\hat{h}_S \approx \mathcal{N}\left(\mathbb{E}[h(\theta)], \frac{\text{Var}[h(\theta)]}{S}\right)$$

- In practice, we do not know $\text{Var}[h(\theta)]$ so we use the sample variance, $\hat{\sigma}^2$.
- $\frac{\hat{\sigma}}{\sqrt{S}}$ is the Monte Carlo Standard Error (MCSE), which quantifies the uncertainty due to using a finite number of samples

Why ordinary Monte Carlo is not enough

- Monte Carlo provides a general *recipe* to compute quantities of interest, but not a general *recipe* for sampling from complex, high-dimensional distributions
- rejection sampling: inefficient for high-dimensional distributions
- importance sampling: can be unreliable when the proposal and target differ substantially

Markov chain Monte Carlo (MCMC)

- A very important family of Monte Carlo methods

Markov chain Monte Carlo (MCMC)

- A very important family of Monte Carlo methods
- Useful to generate samples from complex distributions (potentially) in higher dimensions

Markov chain Monte Carlo (MCMC)

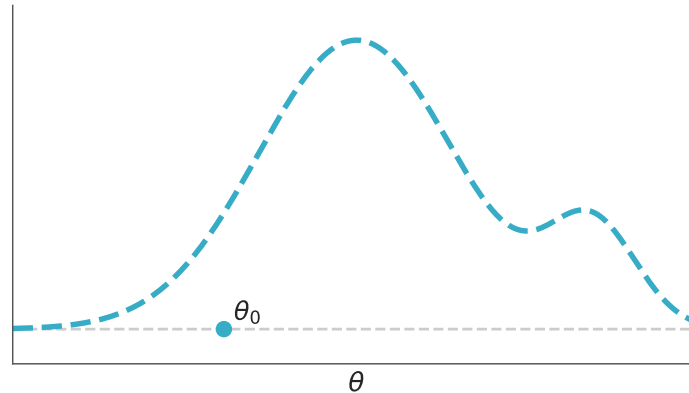
- A very important family of Monte Carlo methods
- Useful to generate samples from complex distributions (potentially) in higher dimensions
- The key idea:
 - generate parameter values from a proposal distribution
 - decide whether to keep it
 - repeat

Markov chain Monte Carlo (MCMC)

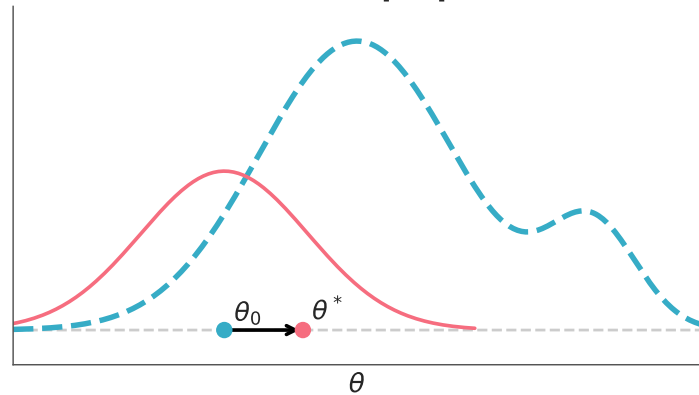
- A very important family of Monte Carlo methods
- Useful to generate samples from complex distributions (potentially) in higher dimensions
- The key idea:
 - generate parameter values from a proposal distribution
 - decide whether to keep it
 - repeat
- There are many MCMC methods:
 - Metropolis–Hastings
 - Gibbs sampling
 - Hamiltonian Monte Carlo (HMC) (next lecture)
 - etc...

Metropolis method

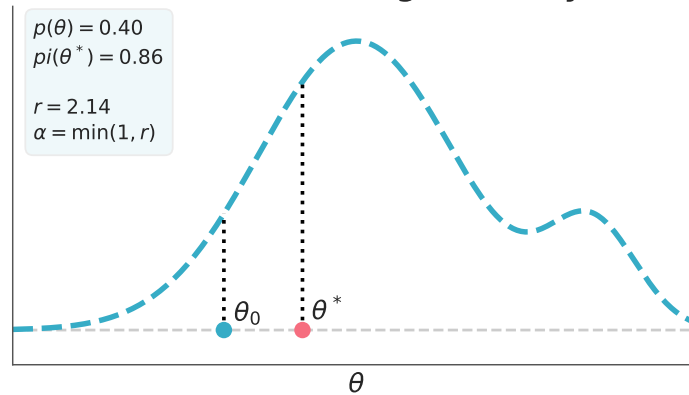
Start



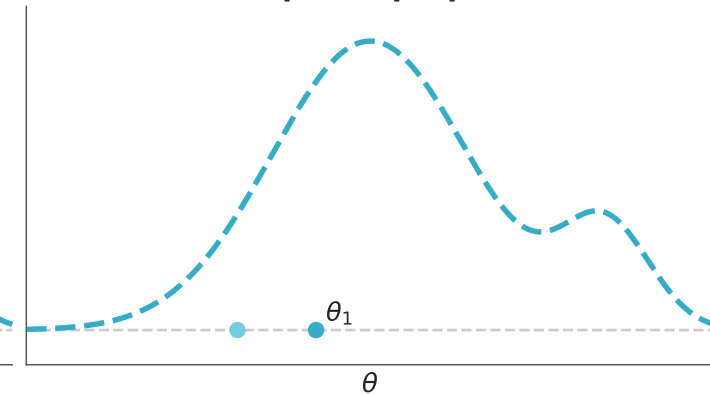
Generate a proposal



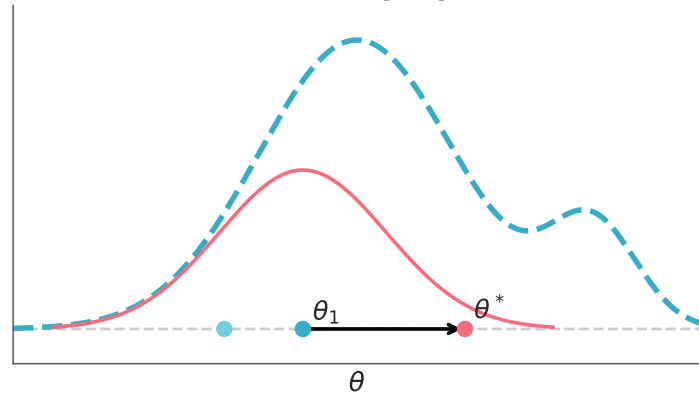
Evaluate the target density



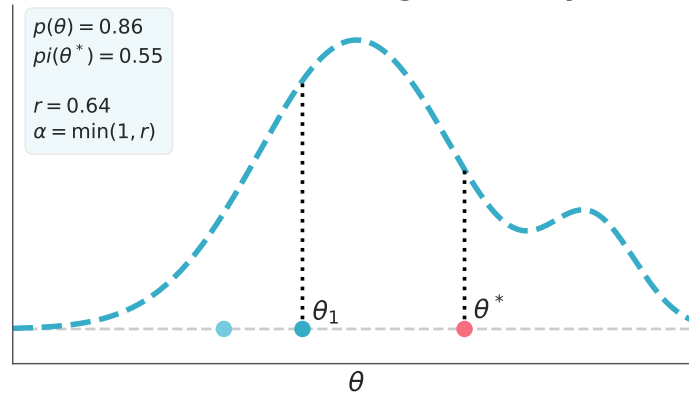
Accept the proposal



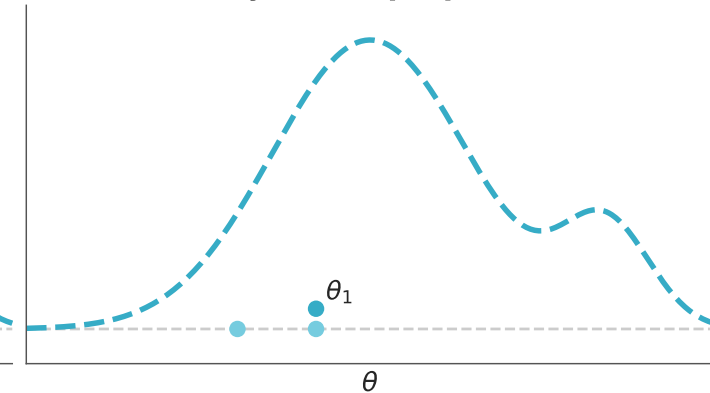
Generate a proposal



Evaluate the target density



Reject the proposal



Metropolis–Hastings

1. Start from some parameter point θ^0
2. At iteration t , propose a new point

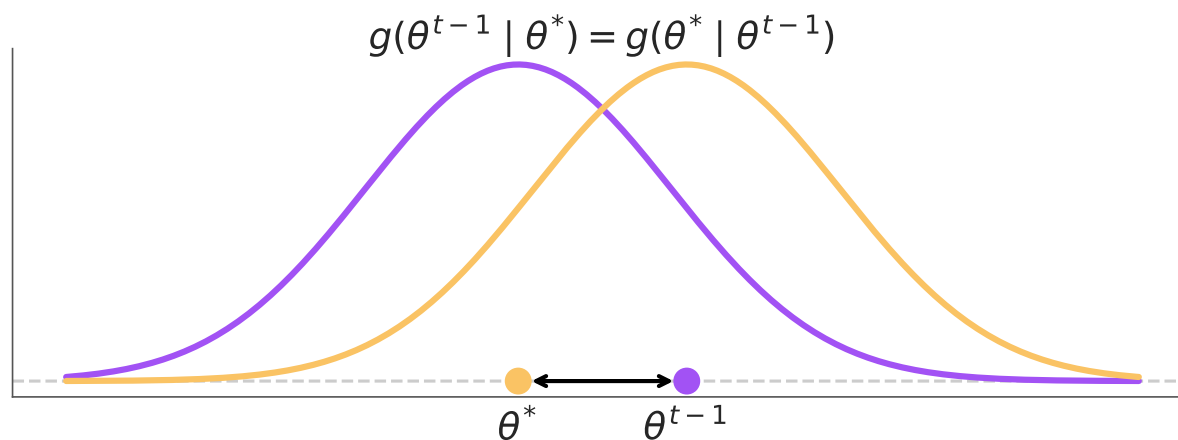
$$\theta^* \sim g(\theta^* \mid \theta^{t-1})$$

3. Evaluate the density of the proposed and current point and accept with probability:

$$\alpha(\theta^{t-1}, \theta^*) = \min \left(1, \frac{p(\theta^* \mid y)}{p(\theta^{t-1} \mid y)} \frac{g(\theta^{t-1} \mid \theta^*)}{g(\theta^* \mid \theta^{t-1})} \right)$$

Accept $\rightarrow \theta^t = \theta^*$; Reject $\rightarrow \theta^t = \theta^{t-1}$

4. Repeat from 2



We don't need the normalizing constant

- The acceptance ratio in Metropolis–Hastings only involves a **ratio** of posterior densities:

$$\alpha(\theta^{t-1}, \theta^*) = \min \left(1, \frac{p(\theta^* | y)}{p(\theta^{t-1} | y)} \frac{g(\theta^{t-1} | \theta^*)}{g(\theta^* | \theta^{t-1})} \right)$$

- By Bayes theorem

$$\frac{p(\theta^* | y)}{p(\theta^{t-1} | y)} = \frac{\frac{p(y | \theta^*)p(\theta^*)}{p(y)}}{\frac{p(y | \theta^{t-1})p(\theta^{t-1})}{p(y)}} = \frac{p(y | \theta^*)p(\theta^*)}{p(y | \theta^{t-1})p(\theta^{t-1})}$$

- The normalizing constant $p(y)$ cancels, so we don't need to compute it!
- More generally, if $p(\theta | y) \propto q(\theta | y)$ for some unnormalized q , the ratio $\frac{q(\theta^* | y)}{q(\theta^{t-1} | y)}$ is all we need.

OK but, why does this work? (Intuition)

- The acceptance ratio compares the density at the proposed point to the density at the current one
- Moving to a more likely point: always accept
- Moving to a less likely point: the ratio tells us how reluctant to be
 - a slightly less likely point $\rightarrow$ accept almost always
 - a much less likely point $\rightarrow$ accept rarely
- Rejecting is not discarding: we stay at θ^{t-1} . So at each iteration we always accumulate a draw.
- As a result, the samples at each value of θ are proportional to $p(\theta \mid y)$
 - The simulation automatically spends more time in regions of higher posterior density

Markov chain

- MH proceeds by generating a sequence of draws

$$\theta^0, \theta^1, \theta^2, \dots$$

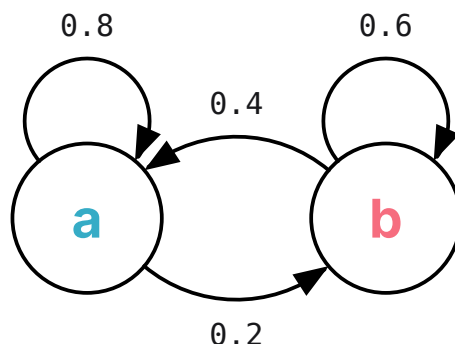
- The present draw depends only on the previous one
- Such sequence can be described as a **Markov chain**
- A sequence of random variables is a Markov chain if

$$p(\theta^t \mid \theta^0, \theta^1, \dots, \theta^{t-1}) = p(\theta^t \mid \theta^{t-1})$$

In words: given the current state, the past does not provide additional information about the next state

Markov chain: a simple example

- Consider a chain with two states, **a** and **b**



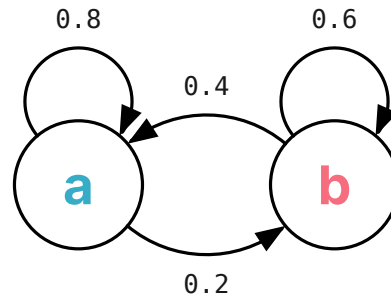
- We can simulate a particular sequence and count how often each state is visited **a a b b a a b b b a a a a a b b a a a a a b...**

$$p(a) \approx \frac{1}{S} \sum_{s=1}^S I(\text{state} = a) = 0.7$$

- This is the same idea as the Monte Carlo, simulate a long sequence of states and count how often we are in state **a**

Markov chain: The transition matrix

- Consider a chain with two states, **a** and **b**



- We can write the transition probabilities in matrix form:

$$T = \begin{pmatrix} & a & b \\ a & 0.8 & 0.2 \\ b & 0.4 & 0.6 \end{pmatrix}$$

- Starting from either state, and repeatedly applying the transition we get

$$T^s = \begin{pmatrix} \mathbf{0.67} & 0.33 \\ 0.67 & \mathbf{0.33} \end{pmatrix}$$

- The long-run probabilities are therefore approximately

$$p(a) = 0.67, \quad p(b) = 0.33$$

Markov chain: stationary distribution

- If $p^{(t)} = (p^{(t)}(a), p^{(t)}(b))$, then

$$p^{(t)} = p^{(t-1)}T$$

- Starting from **a**:

$$p^{(1)} = (0.8, 0.2) \quad p^{(2)} = (0.72, 0.28) \quad \dots \quad p^{(t)} \rightarrow (0.67, 0.33)$$

- No matter where we start, $p^{(t)}$ converges to the same distribution $\pi = (0.67, 0.33)$
- π is called the stationary distribution: once reached, the chain stays there

$$\pi T = \pi$$

Detailed balance

- Checking $\pi T = \pi$ directly is often hard for complex chains
- A simpler and sufficient condition: detailed balance

$$\pi(a) T(a \rightarrow b) = \pi(b) T(b \rightarrow a)$$

the flow from a to b exactly balances the flow from b to a

- If detailed balance holds for all states then π is a stationary distribution
- We can check this directly: with $\pi = (0.67, 0.33)$,

$$\pi(a) T(a \rightarrow b) = 0.67 \times 0.2 = 0.133$$

$$\pi(b) T(b \rightarrow a) = 0.33 \times 0.4 = 0.133$$

balanced!

Back to Metropolis–Hastings

- MH constructs a Markov chain over θ :
 - The transition probabilities are given by the proposal g and the acceptance rule α
 - We want $p(\theta \mid y)$ to be the stationary distribution
- To prove that's true we have to establish:
 1. The sequence $\theta^{(0)}, \theta^{(1)}, \dots$ is a Markov chain with a **unique** stationary distribution
 2. That stationary distribution is $p(\theta \mid y)$
- We'll take (1) as given (next slide) and then focus on proving (2) using **detailed balance**

Uniqueness of the stationary distribution

- For a Markov chain to have a **unique** stationary distribution, it must be:
 - irreducible: positive probability of eventually reaching any state from any other
 - aperiodic: doesn't cycle through states on a fixed schedule
 - recurrent: probability of returning to any given state is 1

Uniqueness of the stationary distribution

- For a Markov chain to have a **unique** stationary distribution, it must be:
 - irreducible: positive probability of eventually reaching any state from any other
 - aperiodic: doesn't cycle through states on a fixed schedule
 - recurrent: probability of returning to any given state is 1
- These hold for MH under mild conditions on the proposal g and the target $p(\theta|y)$:
 - **Irreducible**: whenever $g(\theta^* | \theta) > 0$, we also have $\alpha(\theta, \theta^*) > 0$, any point reachable by the proposal is reachable by the chain.
 - **Aperiodic**: As we can always stay at the current state a "return" after 1 step is always possible, and then returns can happen after any number of steps.
 - **Recurrent**: If $p(\theta|y)$ integrates to 1, the chain can't drift off to low-density regions forever, it's pulled back to any given state in finite expected time

Why Metropolis–Hastings works

- Take two points θ_a and θ_b
- When moving from θ_a to θ_b we have

$$p(\theta_a|y) \cdot g(\theta_b|\theta_a) \cdot \overbrace{\min\left(1, \frac{p(\theta_b|y)g(\theta_a|\theta_b)}{p(\theta_a|y)g(\theta_b|\theta_a)}\right)}^{\alpha(\theta_a, \theta_b)}$$

- Write $L = p(\theta_a|y)g(\theta_b|\theta_a)$ and $R = p(\theta_b|y)g(\theta_a|\theta_b)$ and substitute L and R into the expression

$$L \cdot \min\left(1, \frac{R}{L}\right)$$

- Assume $L \leq R$, then $\frac{R}{L} \geq 1$ and $\min\left(1, \frac{R}{L}\right) = 1$, giving

$$p(\theta_a|y) \cdot g(\theta_b|\theta_a) \cdot \alpha(\theta_a, \theta_b) = L$$

Why Metropolis–Hastings works

- Now when moving from θ_b to θ_a we have

$$p(\theta_b|y) \cdot g(\theta_a|\theta_b) \cdot \overbrace{\min\left(1, \frac{p(\theta_a|y)g(\theta_b|\theta_a)}{p(\theta_b|y)g(\theta_a|\theta_b)}\right)}^{\alpha(\theta_b, \theta_a)}$$

- Substitute L and R into the expression

$$R \cdot \min\left(1, \frac{L}{R}\right)$$

- Since $L \leq R$, then $\frac{L}{R} \leq 1$ and $\min\left(1, \frac{L}{R}\right) = \frac{L}{R}$

$$p(\theta_b|y) g(\theta_a|\theta_b) \alpha(\theta_b, \theta_a) = R \cdot \left(\frac{L}{R}\right) = L$$

- Both terms are equal, hence detailed balance holds:

$$p(\theta_a|y) g(\theta_b|\theta_a) \alpha(\theta_a, \theta_b) = L = p(\theta_b|y) g(\theta_a|\theta_b) \alpha(\theta_b, \theta_a)$$

- If instead $R \leq L$, the same argument with θ_a, θ_b swapped applies

Markov chain Monte Carlo (recap)

- Produce draws θ^t , given θ^{t-1} , from a Markov chain, constructed so that the equilibrium distribution is $p(\theta \mid y)$
 - [+] This is a very generic algorithm (inference engine)
 - [+] We just need an easy to sample distribution (proposal) and be able to evaluate the target distribution up to a normalizing constant
 - [+] asymptotically, the chain spends the $\alpha\%$ of time where $\alpha\%$ posterior mass is
 - [-] draws are dependent
 - [-] construction of efficient Markov chains is not always easy

Markov chain and expectations

- Andrey Markov proved weak law of large numbers and central limit theorem for certain dependent-random sequences, which were later named Markov chains
 - CLT still holds for Markov chains if the variance is finite
- The probability of each event depends only on the state attained in the previous event (or finite number of previous events)
- Markov estimated the transition probabilities for the 20 000 first vowels and consonants in Pushkin's novel "Yevgeniy Onegin"



MCMC draws are dependent

- Monte Carlo estimates still valid (CLT holds as proved by Markov)

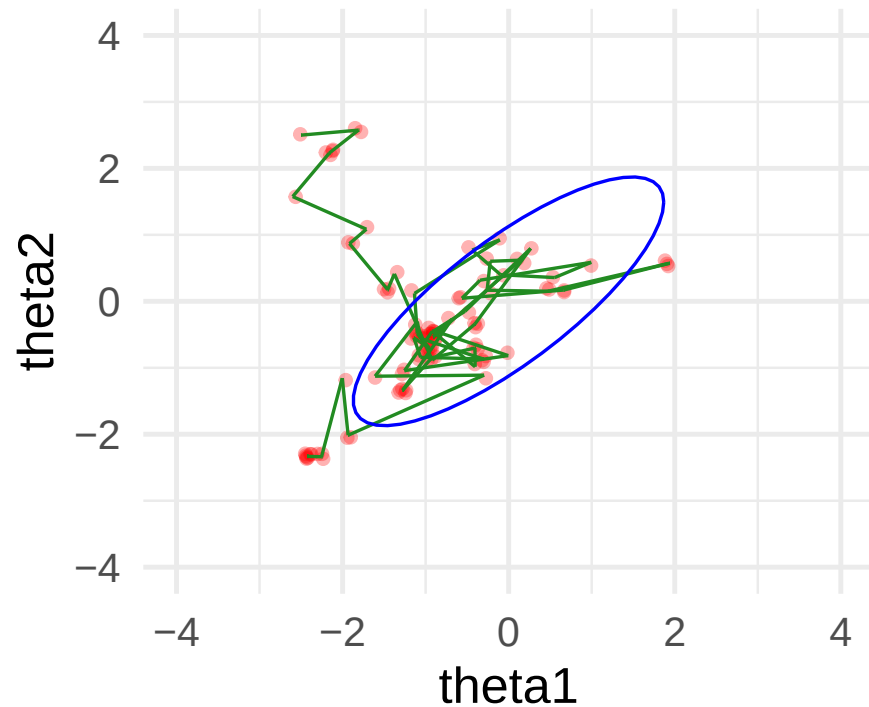
$$E_{p(\theta | y)}[h(\theta)] \approx \frac{1}{S} \sum_{s=1}^S h(\theta^s)$$

- Estimation of Monte Carlo error is more difficult
 - dependency reduces the efficiency
 - we need to compute the effective sample size (ESS)
 - given finite variance, the distribution of the expectation approaches normal distribution with variance $\frac{\sigma_{\theta}^2}{\text{ESS}}$

Metropolis–Hastings: Toy example

- Posterior is a Bivariate Normal

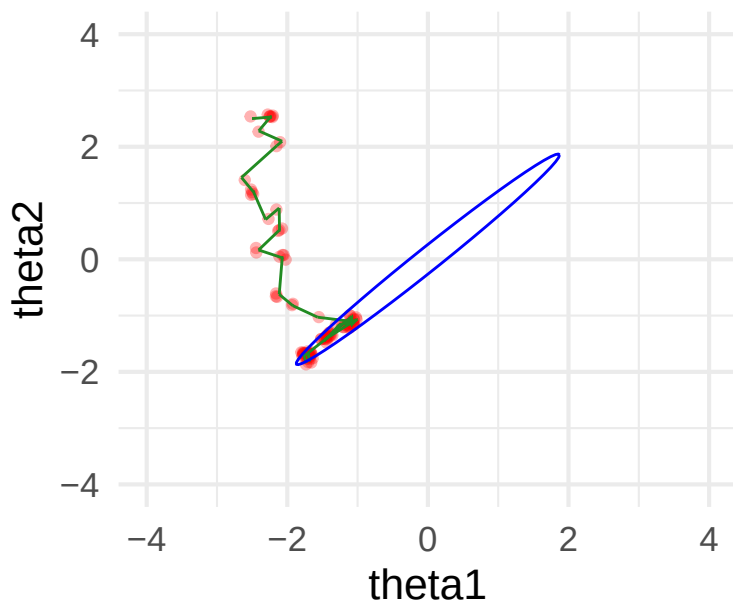
$$\begin{pmatrix} \theta_1 \\ \theta_2 \end{pmatrix} \mid y \sim N\left(\begin{pmatrix} y_1 \\ y_2 \end{pmatrix}, \begin{pmatrix} 1 & \rho \\ \rho & 1 \end{pmatrix}\right)$$



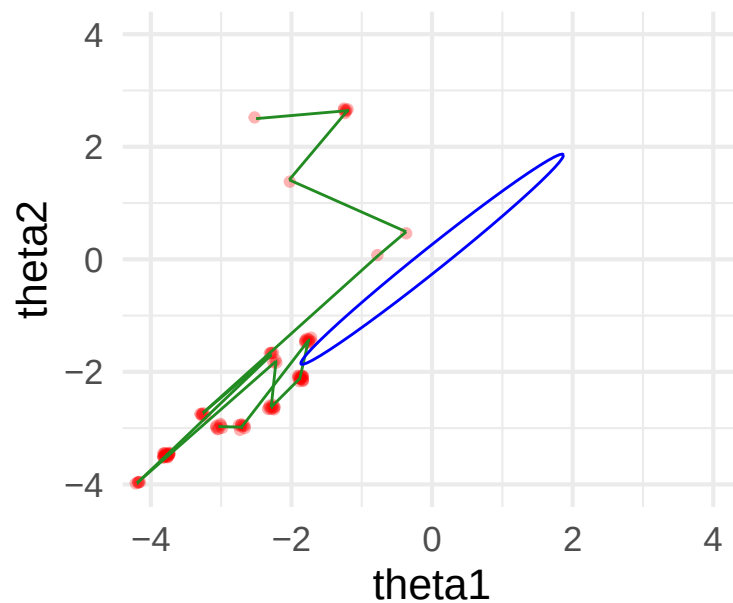
• Draws — Steps of the sampler — 90% HPD

The proposal affects the performance

- In theory: we can use (almost) any proposal distribution and still obtain samples from the target distribution
- In practice: the choice of the proposal distribution greatly affects the efficiency of the sampling



• Draws — Steps of the sampler — 90% HPD



• Draws — Steps of the sampler — 90% HPD

In practice the proposal matters

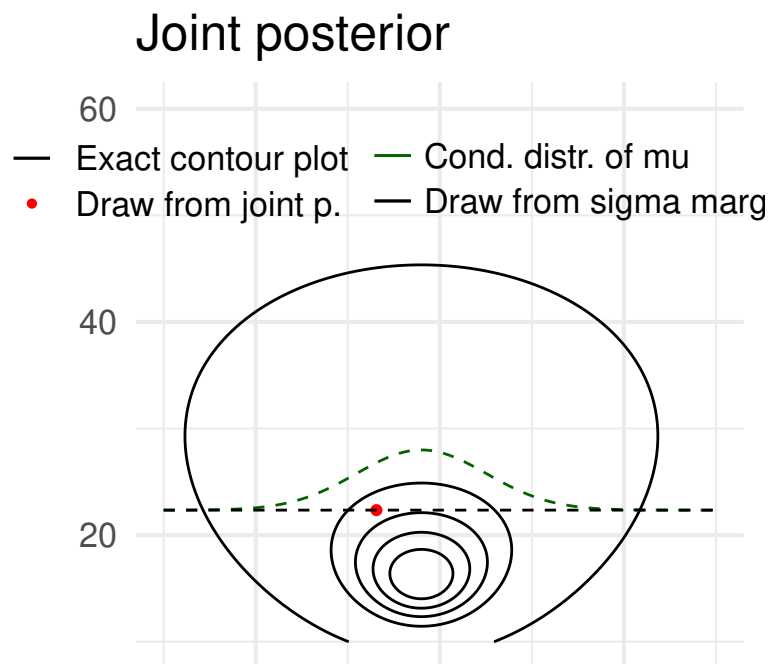
- Ideal proposal distribution is the distribution itself
 - acceptance probability is 1
 - independent draws
 - not usually feasible
- Good proposal distribution resembles the target distribution
 - generic algorithms uses normal or t distribution
- Selecting a good scale for the proposal distribution is crucial
 - small scale $\rightarrow$ many steps accepted, but the chain moves slowly
 - large scale $\rightarrow$ long steps proposed, but many rejected and again chain moves slowly
- Generic rule for acceptance rate is $\approx 10 - 40\%$, it varies with the dimensionality of the posterior and specificities of the MCMC algorithm

From MH to Gibbs

- What if every proposed value were always accepted?
- Model with parameters μ and σ^2 ; alternate between

$$p(\mu \mid \sigma^2, y) \quad \text{and} \quad p(\sigma^2 \mid \mu, y)$$

- If both conditionals are easy to sample from, we build the sequence with no rejection step at all



Gibbs sampling

- Recall the Metropolis–Hastings acceptance probability:

$$\alpha(\theta^{t-1}, \theta^*) = \min \left(1, \frac{p(\theta^* | y) \textcolor{red}{g}(\theta^{t-1} | \theta^*)}{p(\theta^{t-1} | y) \textcolor{teal}{g}(\theta^* | \theta^{t-1})} \right)$$

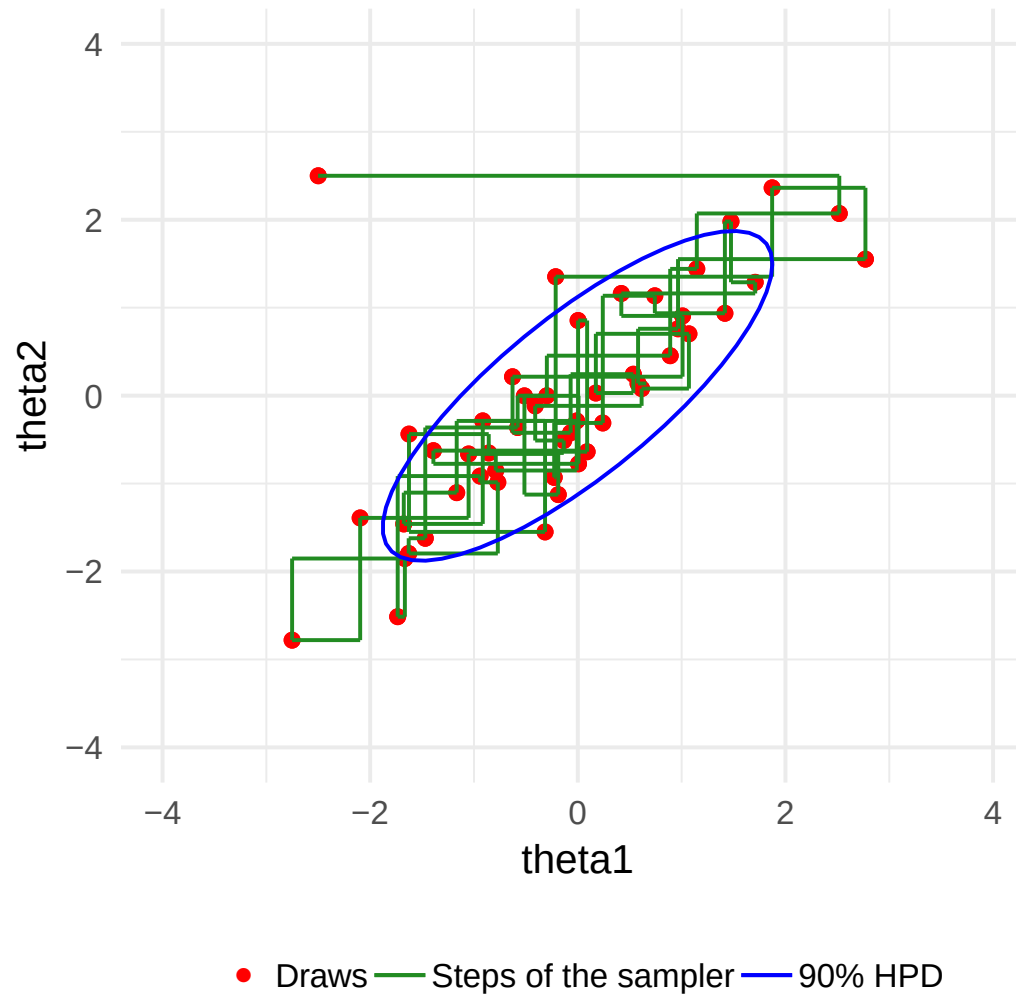
- In Gibbs sampling, the component θ_j is updated conditional on the rest, θ_{-j} (that remain fixed), then

$$\textcolor{teal}{g}(\theta^* | \theta^{t-1}) = p(\theta_j^* | \theta_{-j}, y) \quad \textcolor{red}{g}(\theta^{t-1} | \theta^*) = p(\theta_j^{t-1} | \theta_{-j}, y)$$

- Substituting into α , numerator and denominator both split into the same two factors:

$$\alpha(\theta^{t-1}, \theta^*) = \min \left(1, \frac{p(\theta_j^* | \theta_{-j}, y) \textcolor{red}{p}(\theta_j^{t-1} | \theta_{-j}, y)}{p(\theta_j^{t-1} | \theta_{-j}, y) \textcolor{teal}{p}(\theta_j^* | \theta_{-j}, y)} \right) = \min(1, 1) = 1$$

Gibbs sampling: example



Gibbs demo

Gibbs sampling

- With *conditionally* conjugate priors, the sampling from the conditional distributions is easy for wide range of models
 - BUGS/WinBUGS/OpenBUGS/JAGS
- No algorithm parameters to tune (cf. proposal distribution in Metropolis algorithm)
- For not so easy conditionals, use e.g. inverse-CDF
- Several parameters can be updated in blocks (*blocking*)
- Slow if parameters are highly dependent in the posterior

MH and Gibbs: pros and cons

- Gibbs sampling is useful when the full conditional distributions are easy to sample from
- But often we do not have full conditionals in closed form
- MH is more general and can be applied (virtually) to any model
- MH needs careful design of the proposal distribution
- In practice the proposal is adaptively tuned for each particular problem

We don't live in asymptopia

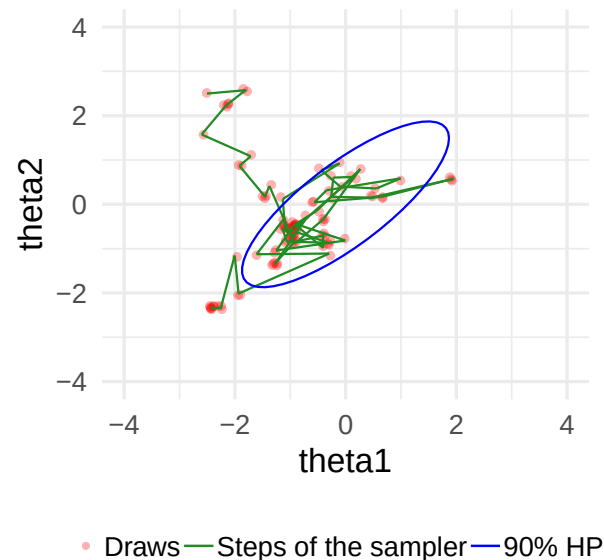
- MCMC has theoretical guarantees only in the limit of infinite draws
- In practice we run finite chains
- We need to check if we can trust our finite chains before computing any posterior summaries from them.

What can go wrong in finite time?

- Non-convergence: the chain hasn't yet found the main mass of the target distribution
- Poor mixing: the chain explores the target slowly (it gets "stuck")

Warm-up and convergence diagnostics

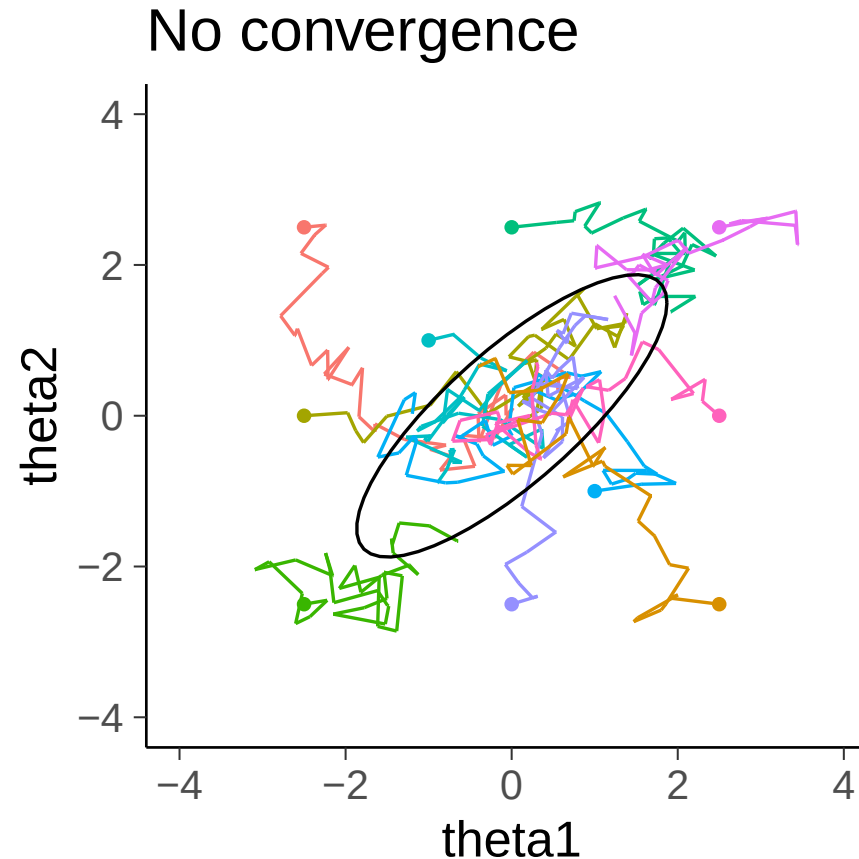
- Asymptotically chain spends the $\alpha\%$ of time where $\alpha\%$ posterior mass is
- but in finite time the initial part of the chain may be non-representative and lower error of the estimate can be obtained by throwing it away



- Warm-up = remove draws from the beginning of the chain
- warm-up may include also phase for adapting algorithm parameters (like the scale of the proposal)

Several chains

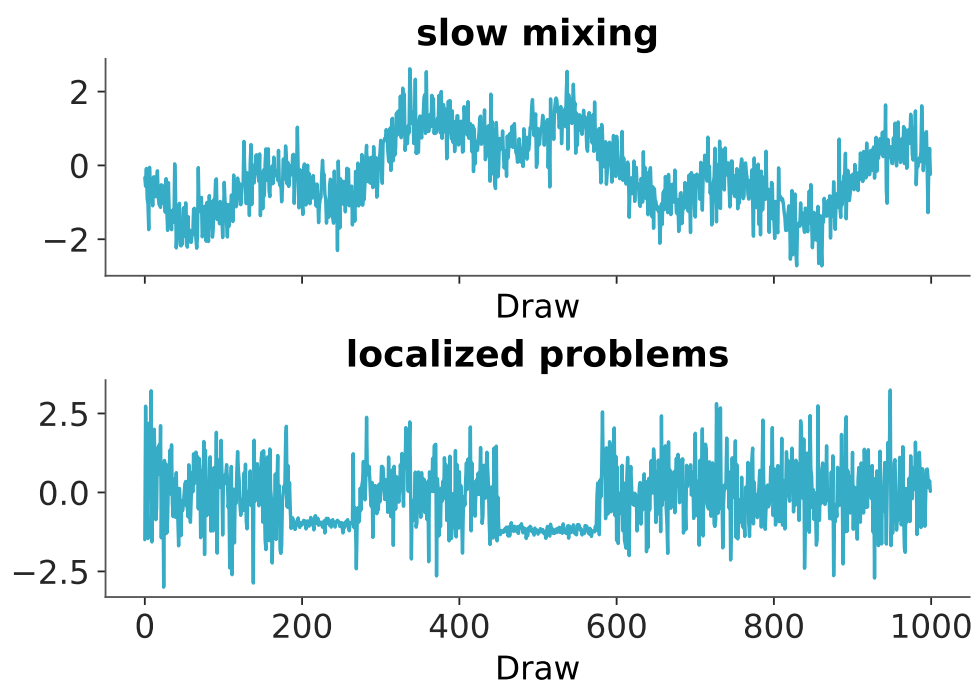
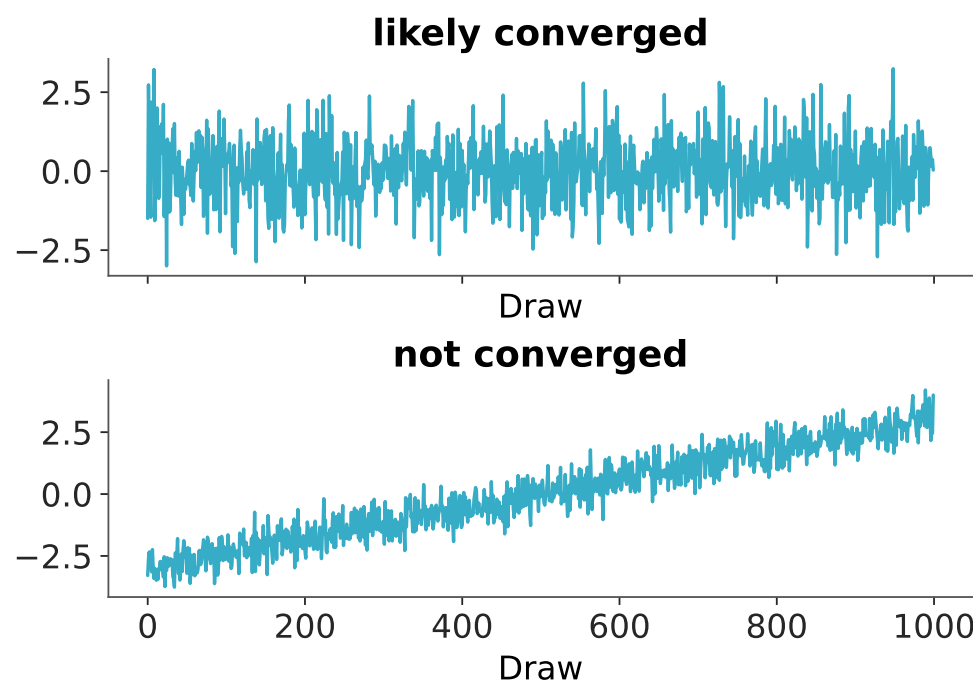
- Start chains from different starting points, preferably overdispersed



- Remove the initial portion of each chain
- Check if the chains are distinguishable from each other

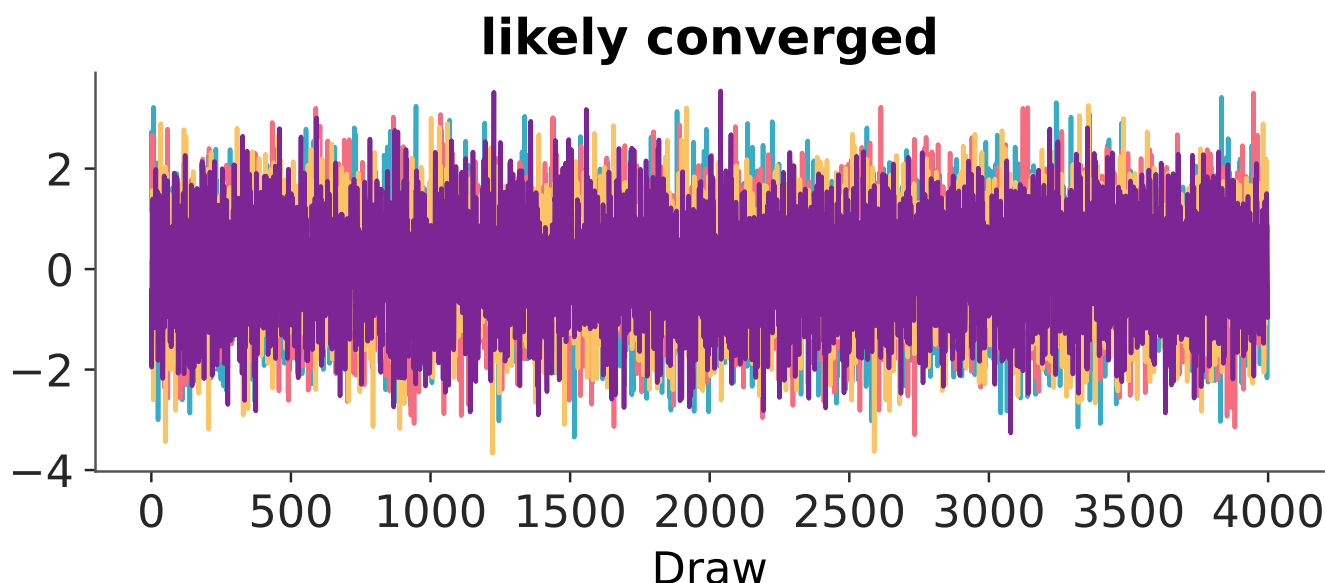
Trace plot: Old school visual check

- Plot the values of the chains over iterations to visually inspect convergence
- It should look "noisy", "fuzzy", "caterpillar-like"
- Overlaying multiple chains helps to visually compare chains
- But overlaying also may hide individual chain behavior



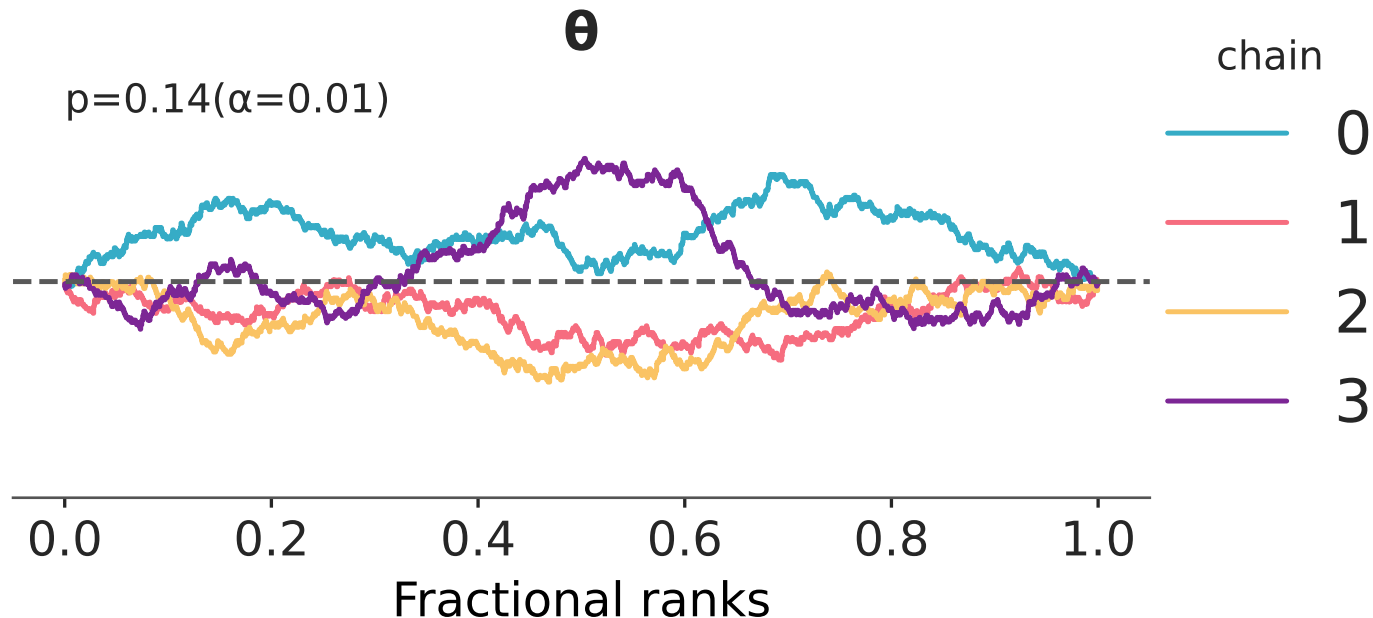
Trace plot: Old school visual check

- Plot the values of the chains over iterations to visually inspect convergence
- It should look "noisy", "fuzzy", "caterpillar-like"
- Overlaying multiple chains helps to visually compare chains
- But overlaying also may hide individual chain behavior



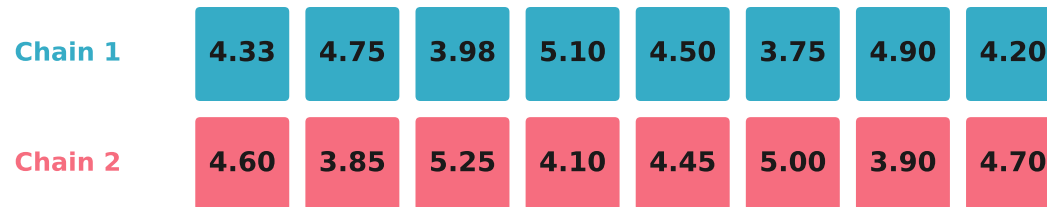
Rank plots: Modern visual check

- If chains have converged to the same distribution, fractional ranks should be uniformly distributed within each chain

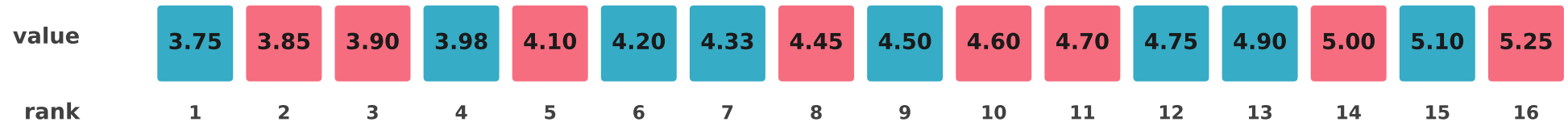


Rank plots: Modern visual check

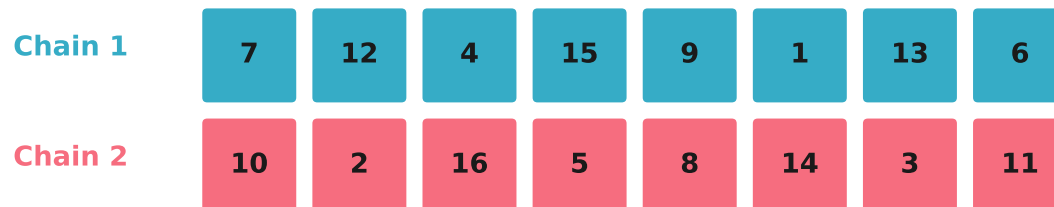
Step 1 - Raw draws from two MCMC chains



Step 2 - Pool both chains together and sort all draws

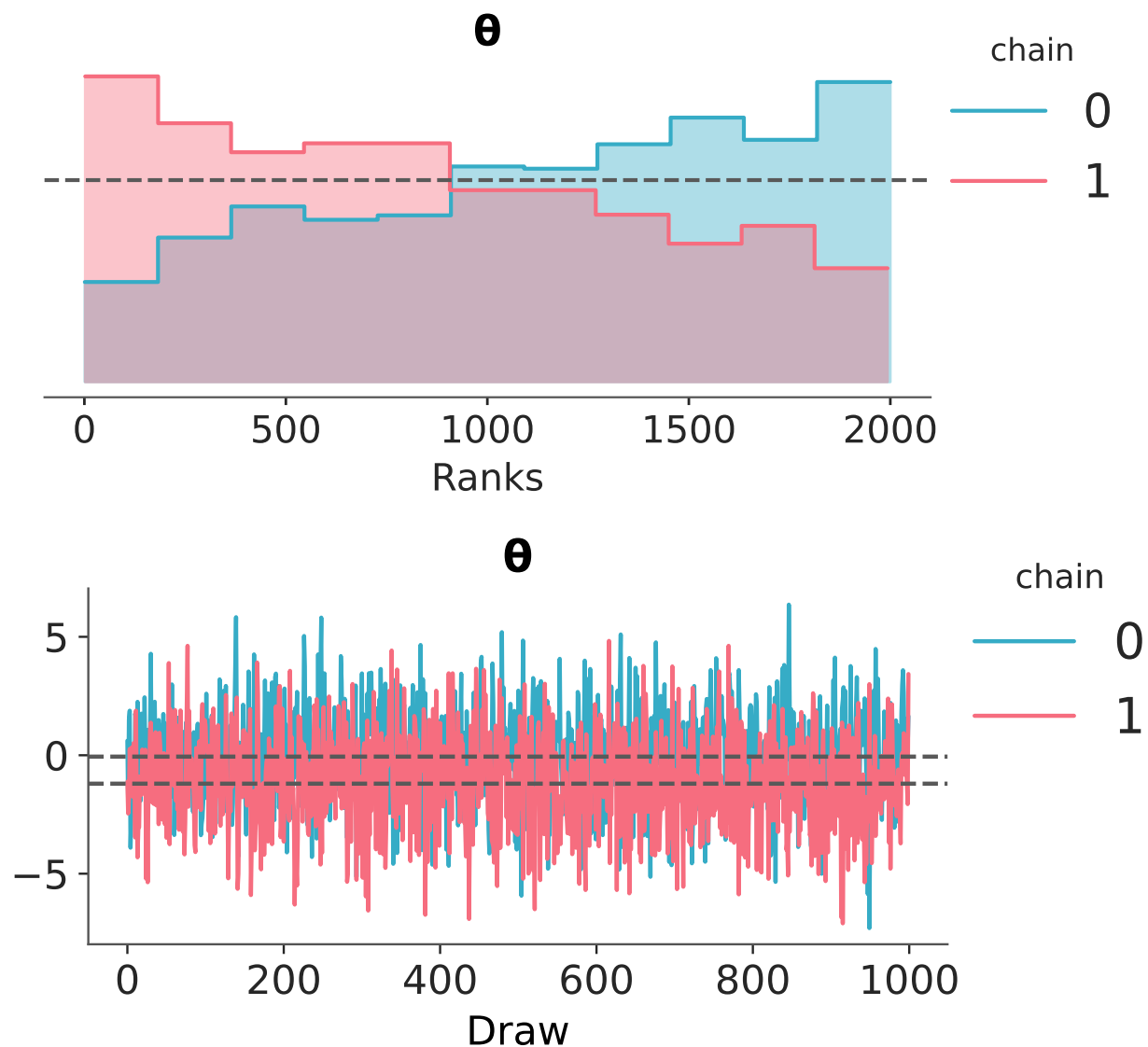


Step 3 — Back to two chains, draws replaced by their rank



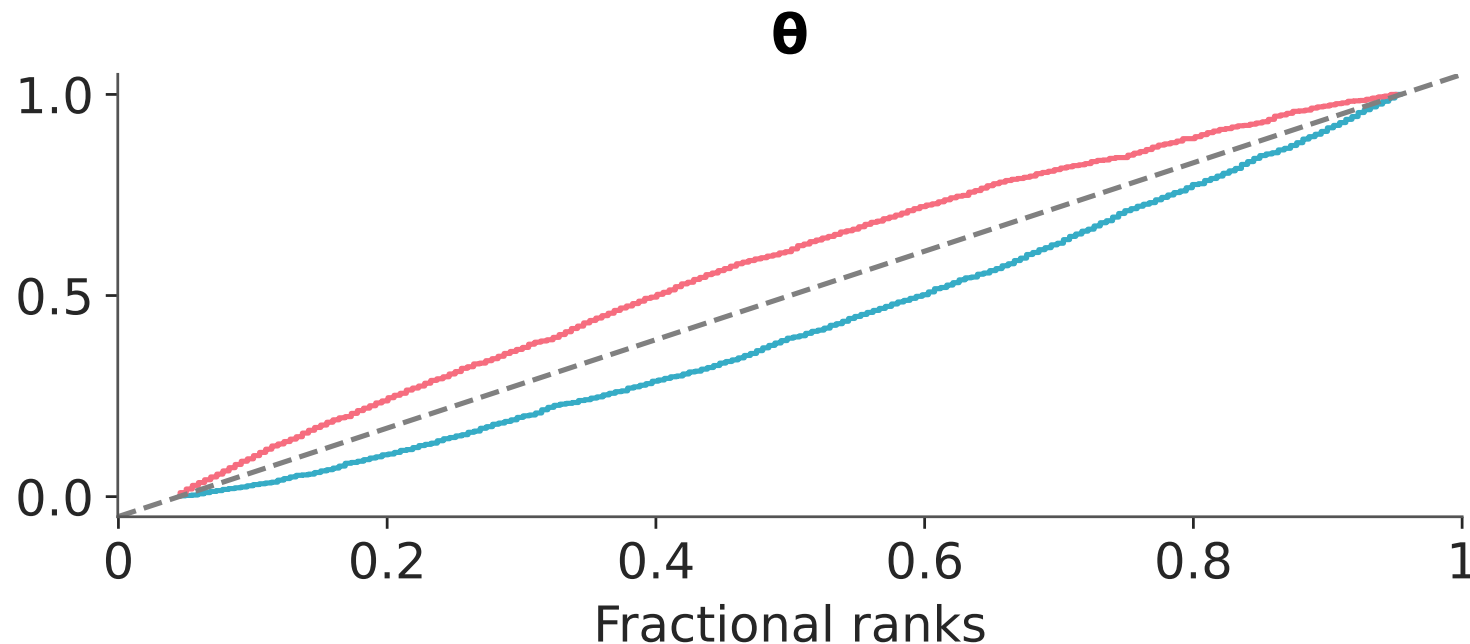
Rank plots: Modern visual check

- Plot the rank distribution within each chain using a histogram



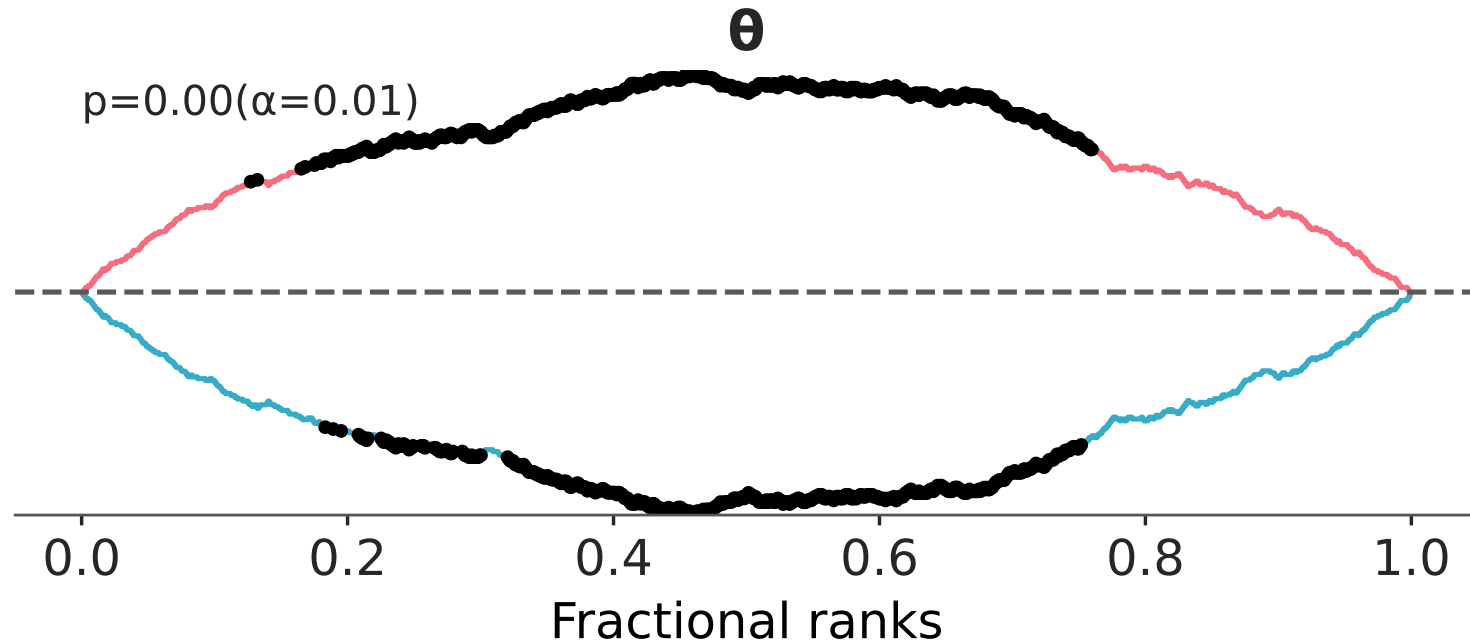
Rank plots: from histograms to ECDFs

- Histograms of ranks can be sensitive to bin choice
- Instead plot the empirical CDF of the ranks
- Use fractional ranks (rescaled to $[0, 1]$) instead of raw ranks (1 to S)
- The CDF of the Uniform(0, 1) goes from (0, 0) to (1, 1) (diagonal line)
- Deviations from the diagonal indicates potential convergence issues



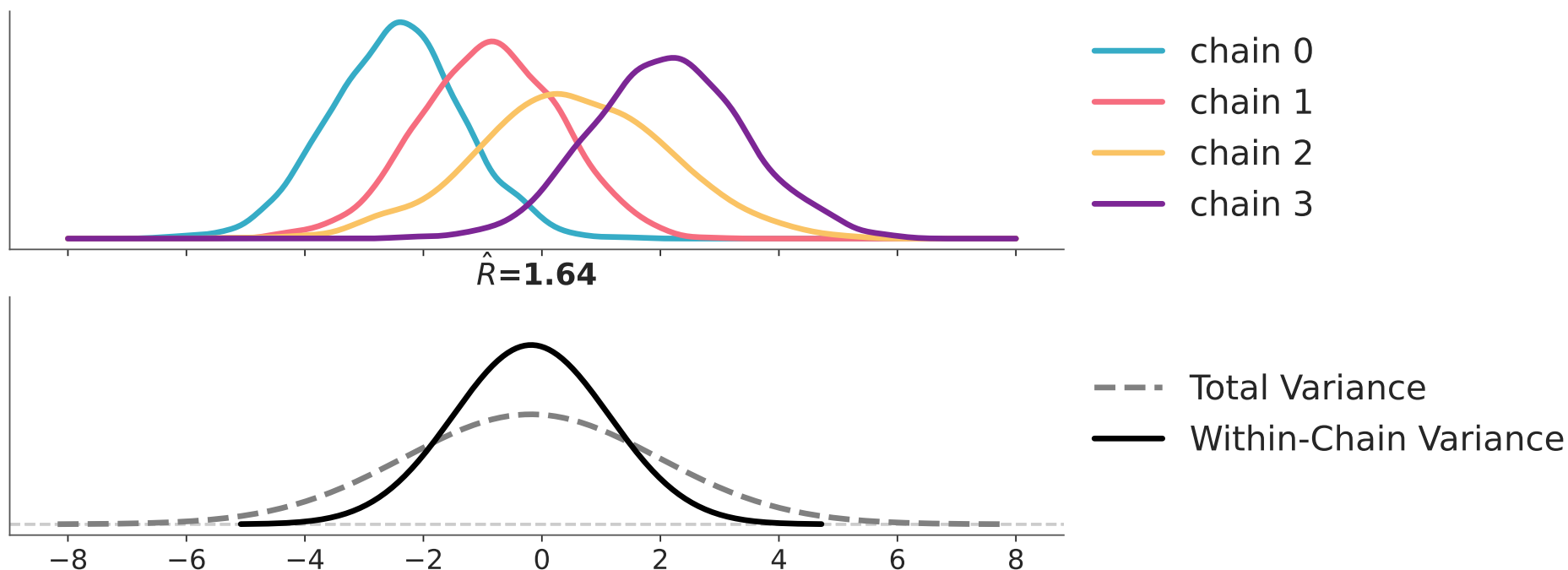
Rank plots: the ECDF difference plot

- Problem: near the diagonal, most of the plot is empty space, deviations may be hard to see
- Fix: plot the difference from the expected line instead
- Now a perfect match is a flat line at 0
- Also add a numerical uniformity check



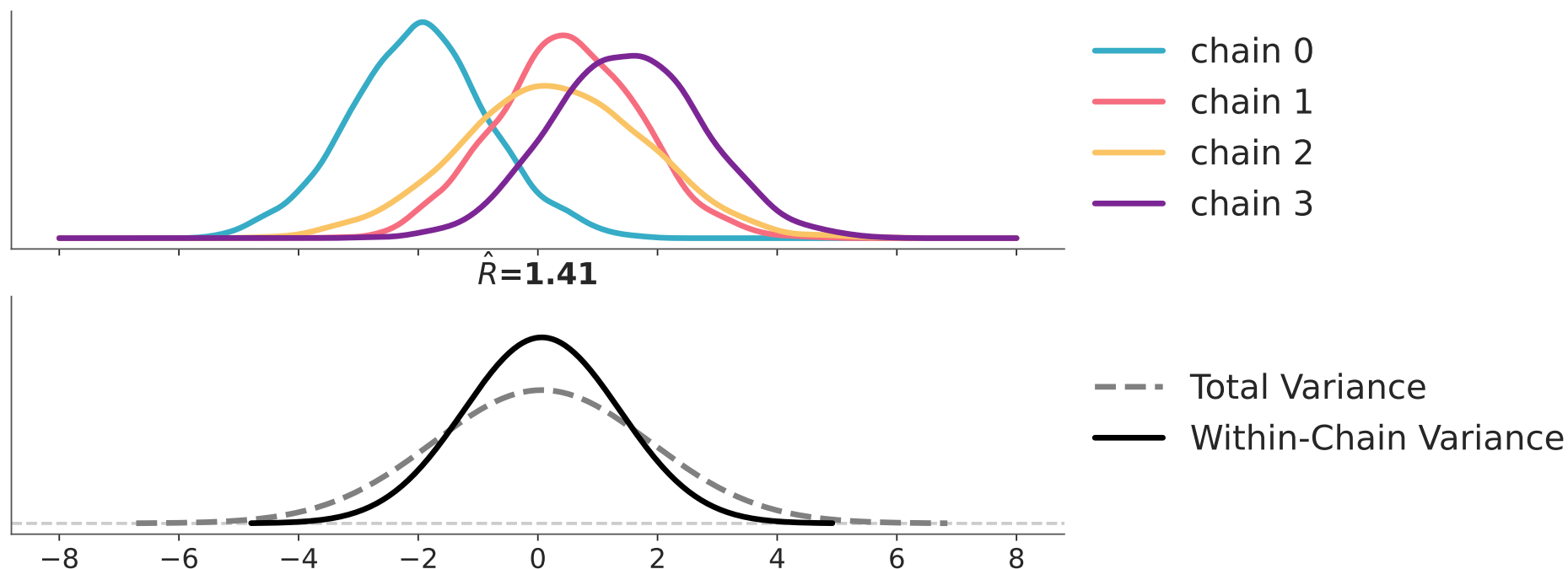
Numerical convergence diagnostic: $\hat{R}$

- Run M chains: if they've converged to the same distribution, they should be indistinguishable
 - Variance **between** chains should match variance **within** each chain
 - If chains haven't converged (e.g. stuck in different regions), between-chain variance will be inflated relative to within-chain variance



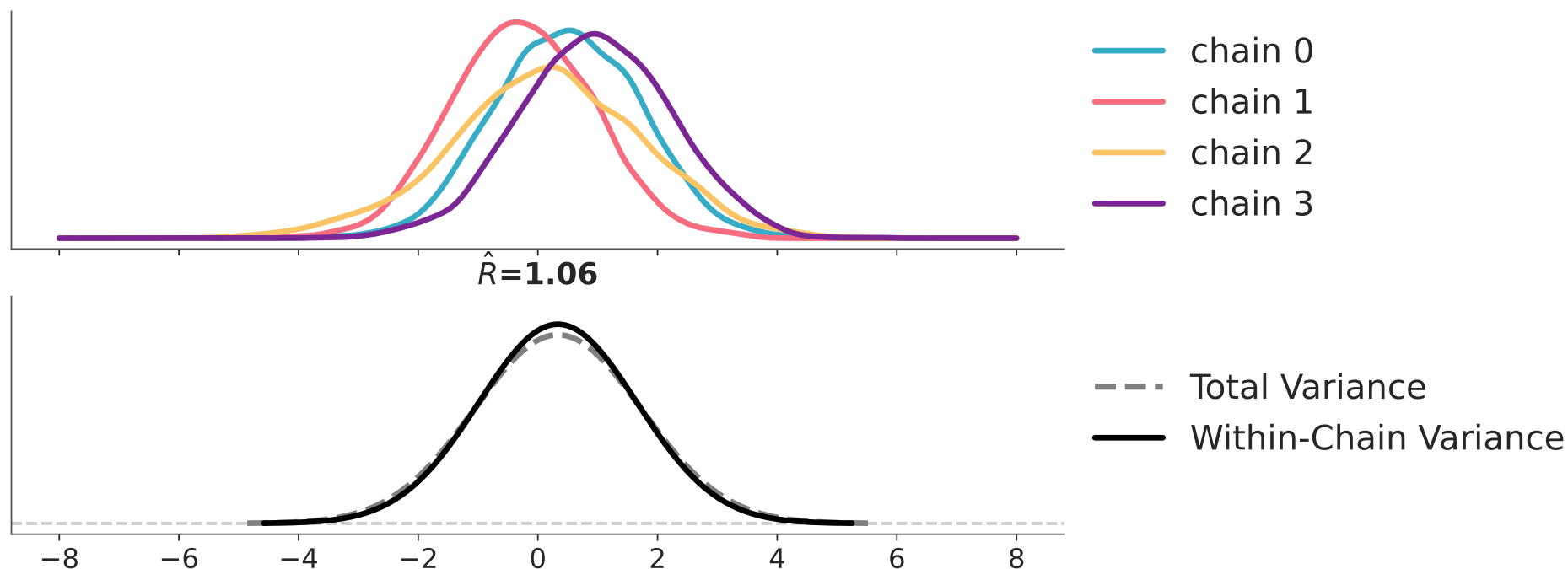
Numerical convergence diagnostic: $\hat{R}$

- Run M chains: if they've converged to the same distribution, they should be indistinguishable
 - Variance **between** chains should match variance **within** each chain
 - If chains haven't converged (e.g. stuck in different regions), between-chain variance will be inflated relative to within-chain variance



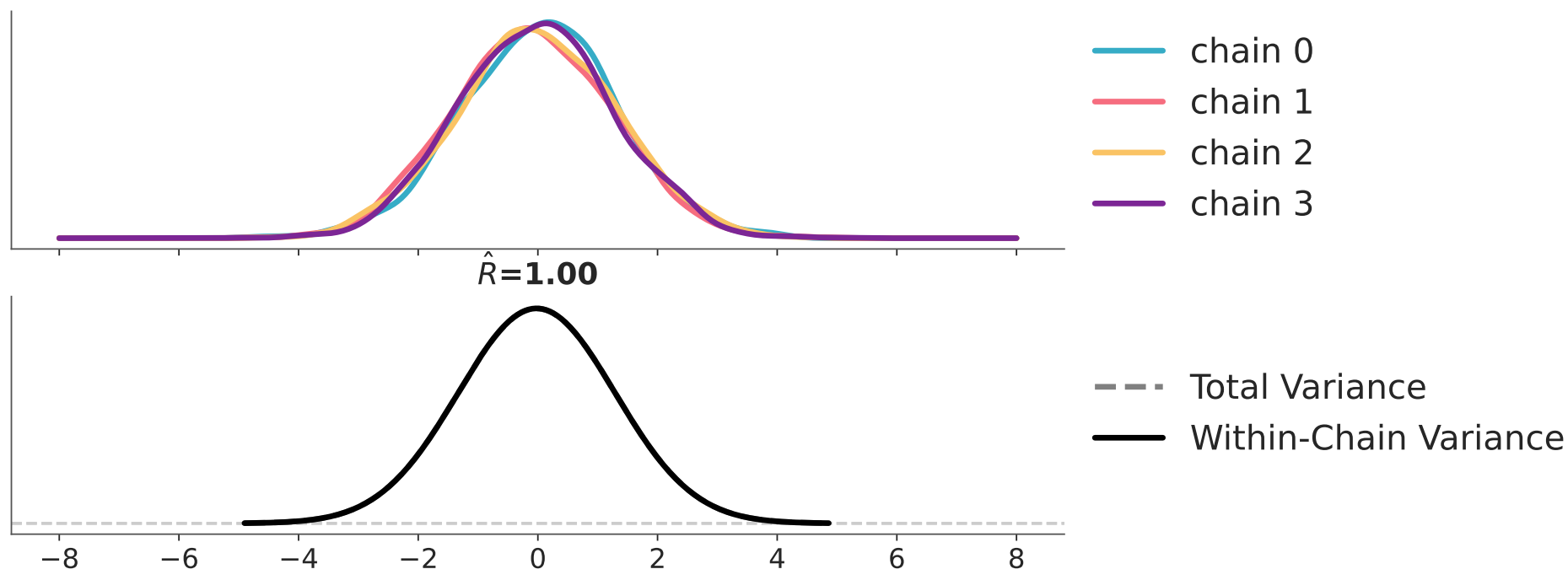
Numerical convergence diagnostic: $\hat{R}$

- Run M chains: if they've converged to the same distribution, they should be indistinguishable
 - Variance **between** chains should match variance **within** each chain
 - If chains haven't converged (e.g. stuck in different regions), between-chain variance will be inflated relative to within-chain variance



Numerical convergence diagnostic: $\hat{R}$

- Run M chains: if they've converged to the same distribution, they should be indistinguishable
 - Variance **between** chains should match variance **within** each chain
 - If chains haven't converged (e.g. stuck in different regions), between-chain variance will be inflated relative to within-chain variance



$\hat{R}$

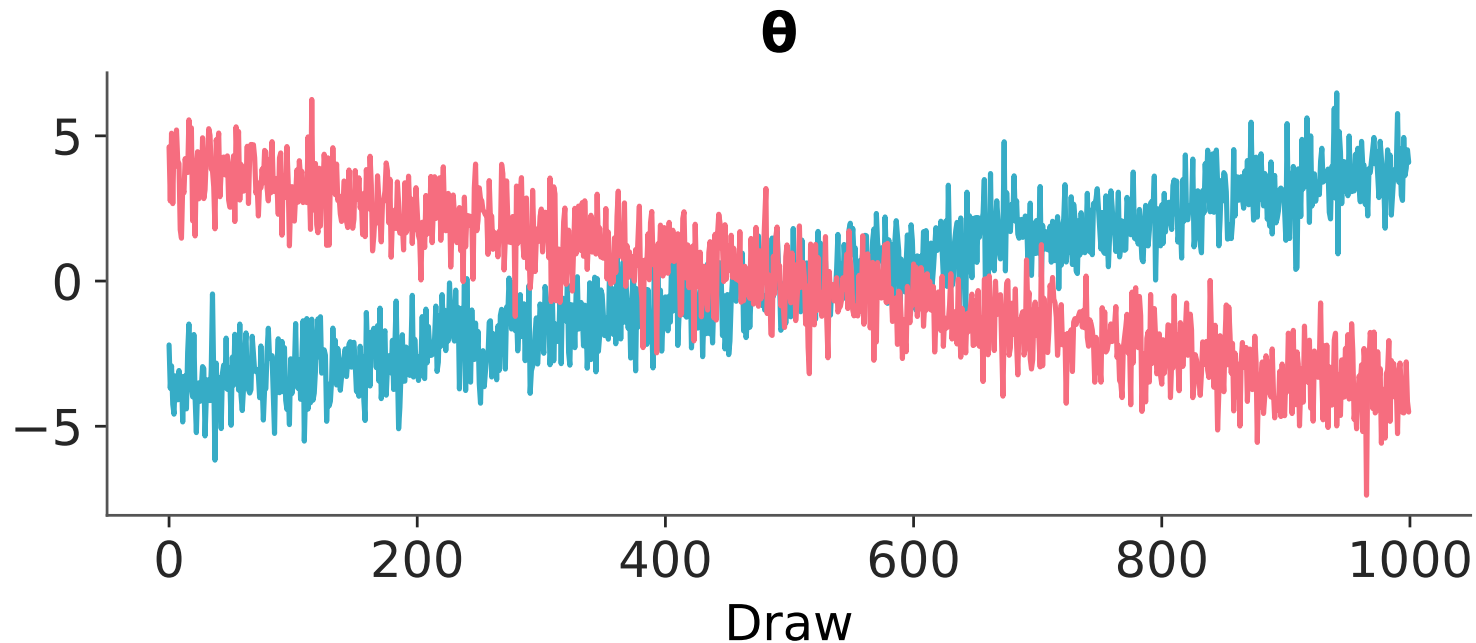
We define $\hat{R}$ as the square root of the ratio of the total variance estimate $\widehat{\text{var}}^+$ to the within-chain variance W :

$$\hat{R} = \sqrt{\frac{\widehat{\text{var}}^+}{W}}$$

- W underestimates the true variance: a chain that hasn't explored everywhere yet looks too narrow.
- $\widehat{\text{var}}^+$ overestimates the true variance if chains started in different places and haven't mixed yet.
- $\hat{R} > 1 \rightarrow$ not converged
- $\hat{R} \approx 1$ likely converged
- This description is very schematic, there are more details in the computation of $\hat{R}$ (and more than one version).

Split- $\hat{R}$

- BDA3: split- $\hat{R}$
- Chains are split in half
 - after splitting, we have M chains, each having N draws
 - this helps to detect non-stationarity within chains



Rank normalized $\hat{R}$

- BDA3 $\hat{R}$ requires finite mean and variance
 - unreliable for heavy tails or finite but very high variance
 - The between-chain variance only compares chain means, so chains with the same location but different scale can look ok

Rank normalized $\hat{R}$

- BDA3 $\hat{R}$ requires finite mean and variance
 - unreliable for heavy tails or finite but very high variance
 - The between-chain variance only compares chain means, so chains with the same location but different scale can look ok
- What is new
 - pool all draws, replace by ranks, and transform with the inverse normal-cdf
 - repeat for the ranks of the absolute difference from the median (folding)
 - report the maximum of the two

Rank normalized $\hat{R}$

- BDA3 $\hat{R}$ requires finite mean and variance
 - unreliable for heavy tails or finite but very high variance
 - The between-chain variance only compares chain means, so chains with the same location but different scale can look ok
- What is new
 - pool all draws, replace by ranks, and transform with the inverse normal-cdf
 - repeat for the ranks of the absolute difference from the median (folding)
 - report the maximum of the two
- Why this helps
 - ranks are always finite and approximately normal if chains have mixed, so the assumptions of $\hat{R}$ hold
 - folding turns differences in scale into differences in location

$\hat{R}$ in practice

- $\hat{R} \approx 1$
- in practice we often look for $\hat{R} < 1.01$
 - If larger more draws could help
 - If too large more severe problems may exist
- Posterior package in R provides functions `rhat_basic()` and `rhat()` to compute $\hat{R}$ and rank normalized $\hat{R}$, respectively.
- ArviZ in Python provides similar functionality `az.rhat(., method="rank")` (default). The function `az.summary()`, also computes $\hat{R}$.

$\hat{R}$ and rank normalized $\hat{R}$ in posterior package

`rhat_basic()` without rank normalization `rhat()` with rank normalization

```
x <- array(data=c(rnorm(1000,mean=-3),
                  rnorm(1000,mean=3)),
           dim=c(1000, 2, 1))
x <- as_draws_matrix(x)
variables(x) <- "N_2"

x |>
  summarise_draws(mean, sd, mcse_mean, rhat_basic, rhat)
  variable    mean    sd mcse_mean rhat_basic  rhat
  N_2         0.0122  3.18      2.15      3.61  1.83
```

$\hat{R}$ and rank normalized $\hat{R}$ in posterior package

`rhat_basic()` without rank normalization `rhat()` with rank normalization

```
x <- array(data=c(rt(1000,df=1)-6,
                    rt(1000,df=1)+6),
           dim=c(1000, 2, 1))
x <- as_draws_matrix(x)
variables(x) <- "t1_2"
x |>
  summarise_draws(mean, sd, mcse_mean, pareto_khat,
                  rhat_basic, rhat)
```

variable	mean	sd	mcse_mean	pareto_khat	rhat_basic	rhat
t1_2	-1.11	42.1	1.23	1.07	1.01	1.47

Effective sample size

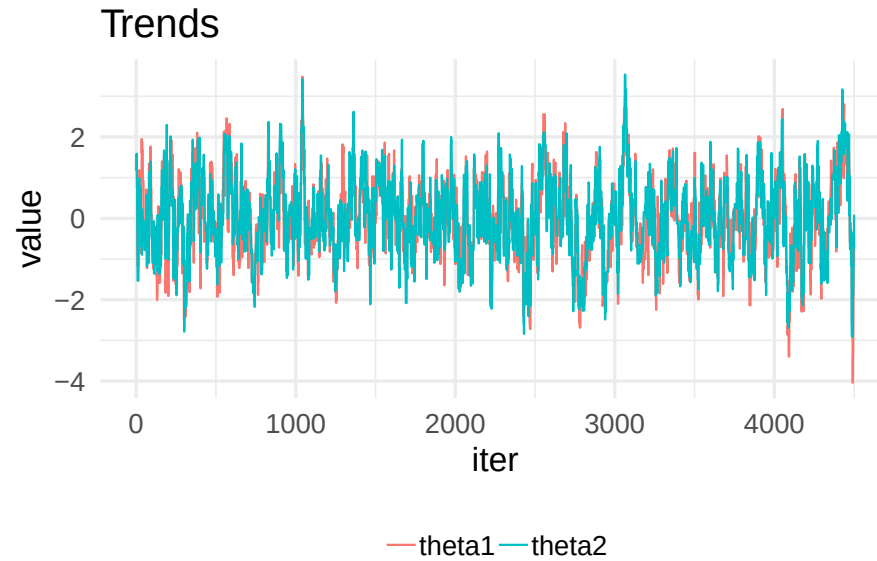
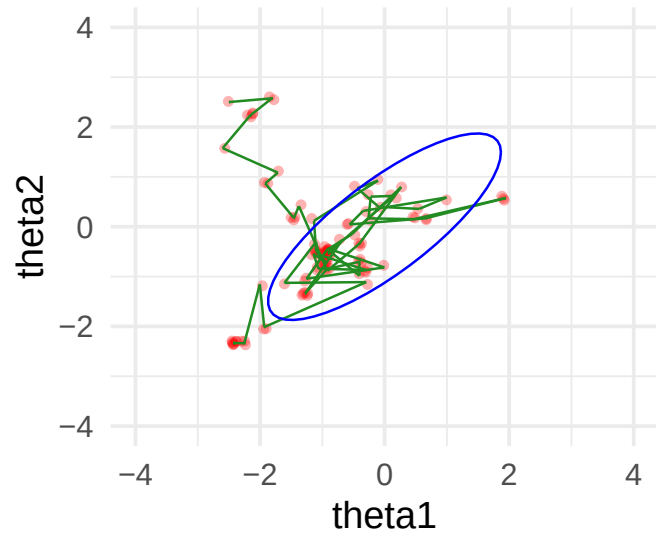
- We can not use the sample size directly and *plug it* into the formula for MCSE
- Draws from MCMC chains are typically correlated, so the effective sample size should be smaller than the total number of draws
- We need to compute the Effective sample size (ESS)
- Once we have the effective sample size, we can use it to compute the Monte Carlo standard error (MCSE) for our estimates.
- It can be estimated using the autocorrelation of the chains.

Time series analysis

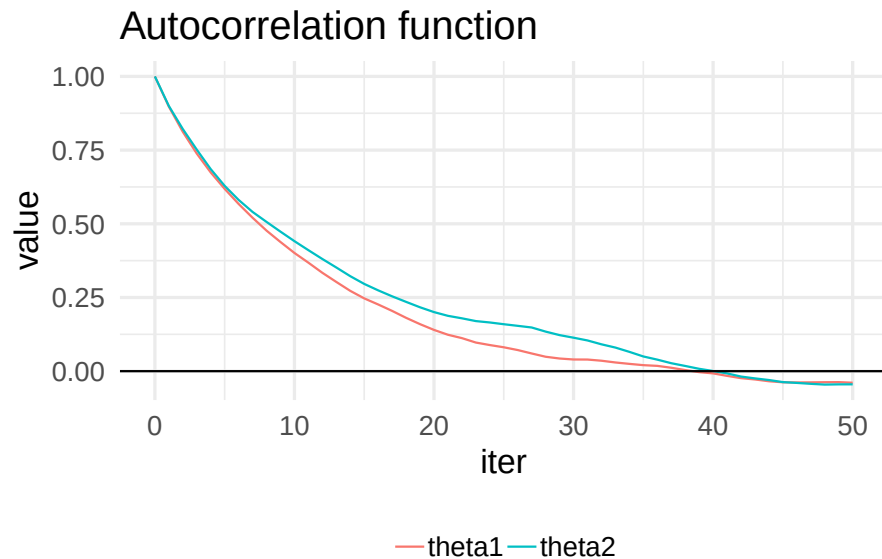
- Autocorrelation function
 - describes the correlation given a certain lag
 - can be used to compare efficiency of MCMC algorithms and parameterizations
 - For real valued, the correlation at lag n

$$\frac{E[(X_{t+n} - E[X])(X_t - E[X])]}{\text{Var}[X]}$$

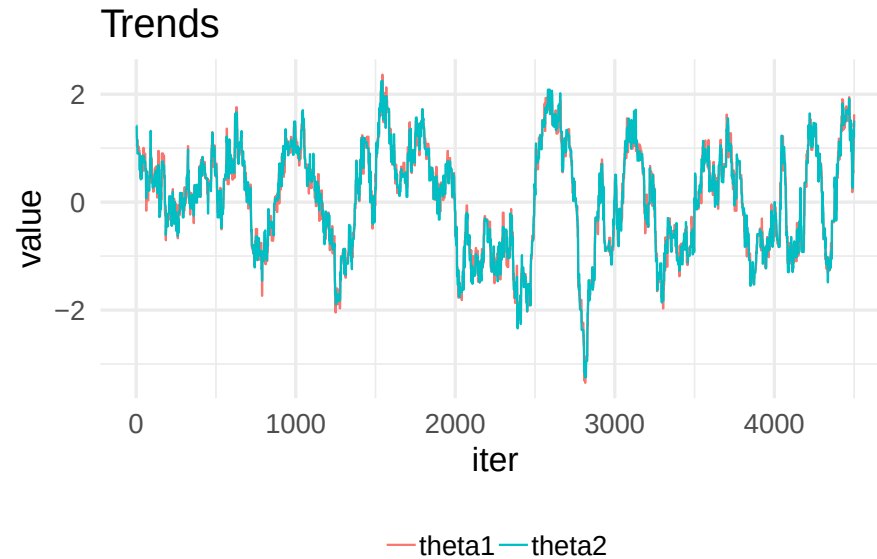
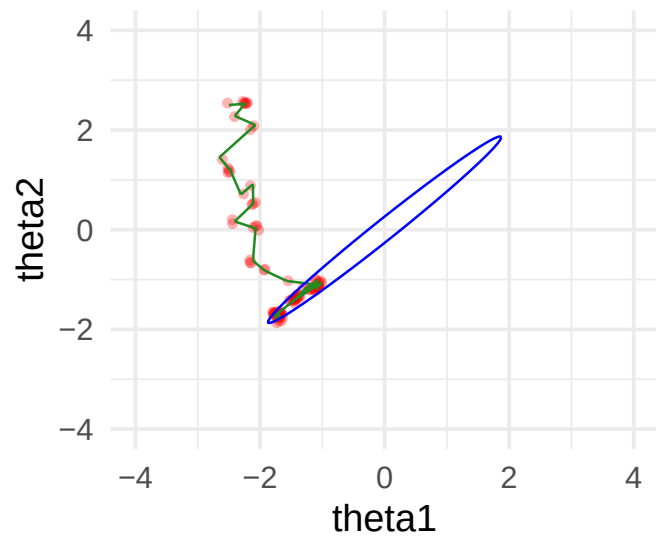
Autocorrelation



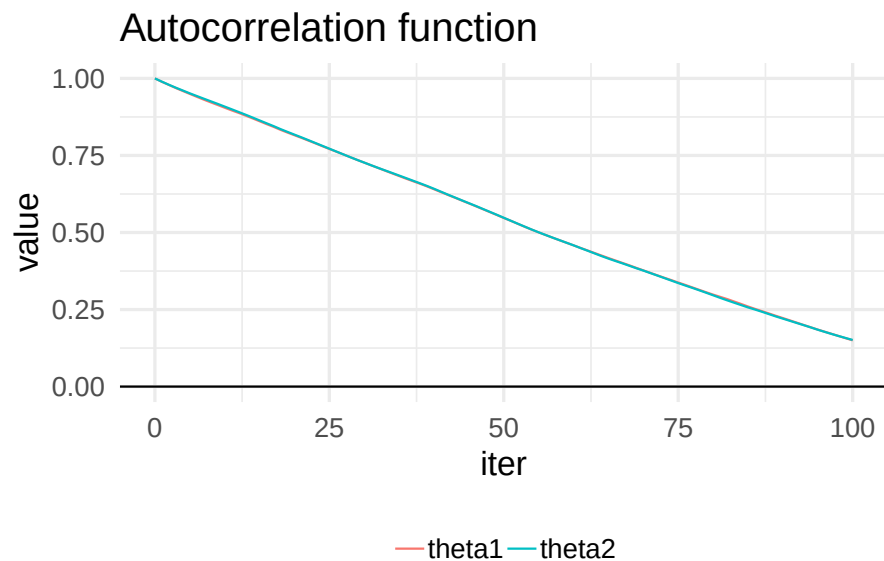
• Draws — Steps of the sampler — 90% HPD



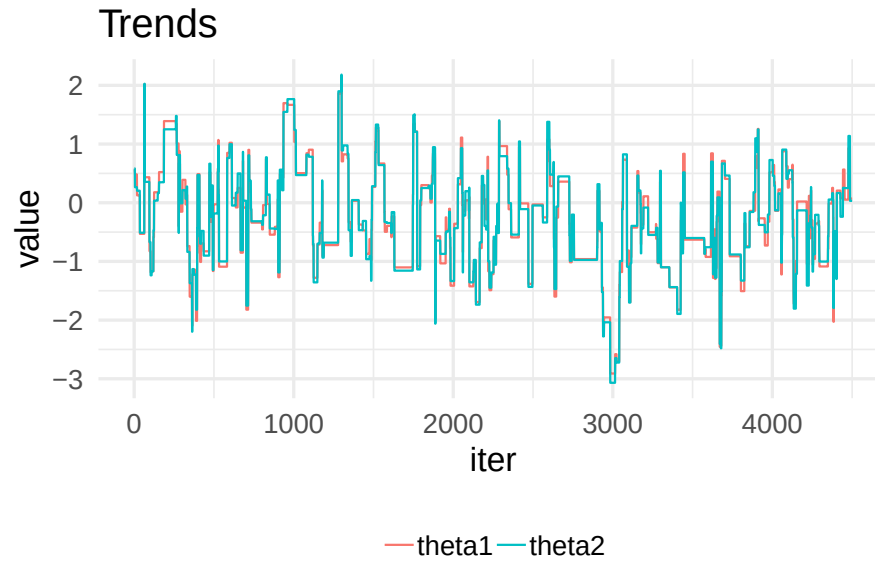
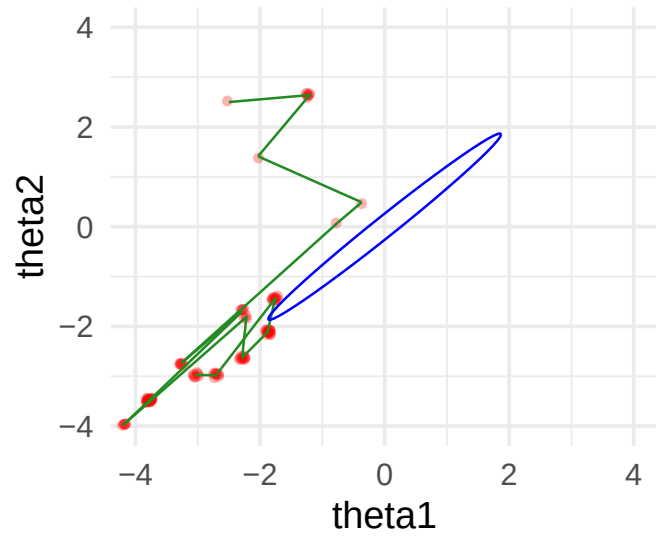
Autocorrelation (slow mixing due to small step size)



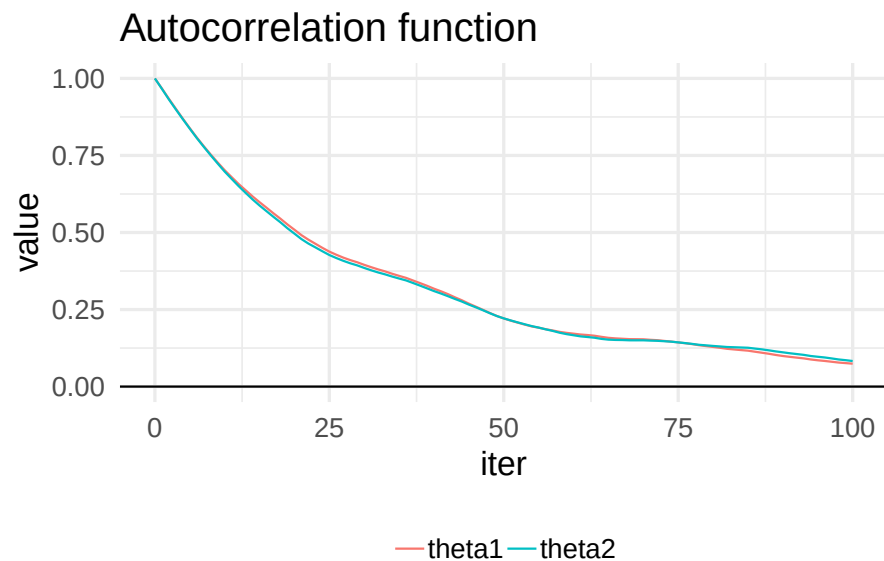
• Draws — Steps of the sampler — 90% HPD



Autocorrelation (slow mixing due to many rejections)



• Draws — Steps of the sampler — 90% HPD



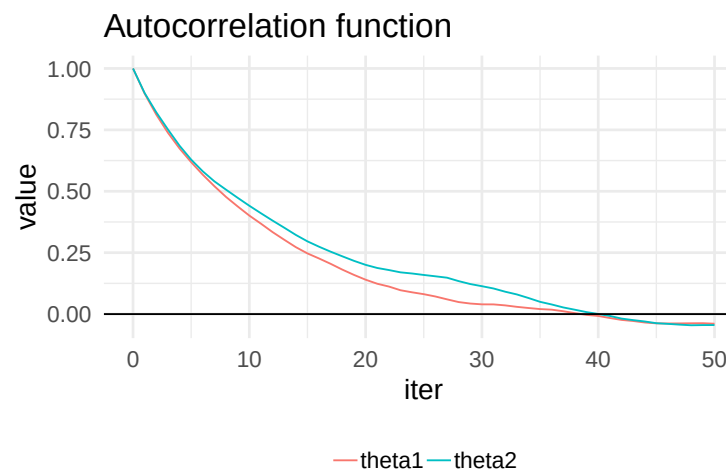
Time series analysis

- Time series analysis can be used to estimate Monte Carlo error in case of MCMC
- For expectation $|\theta|$

$$\text{Var}[|\theta|] = \frac{\sigma_{\theta}^2}{S_{\text{eff}}}$$

where $S_{\text{eff}} = S/\tau$ (=ESS)

- τ is the sum of autocorrelations and describes the relative inefficiency due to the dependency



Time series analysis

- Estimation of the autocorrelation using several chains

$$\hat{\rho}_n = 1 - \frac{\mathbf{W} - \frac{1}{M} \sum_{m=1}^M \hat{\rho}_{n,m}}{2\widehat{\mathbf{var}}^+}$$

where $\hat{\rho}_{n,m}$ is autocorrelation at lag n for chain m , and $\mathbf{W}$ and $\widehat{\mathbf{var}}^+$ are the same as in $\hat{R}$ (without rank normalization)

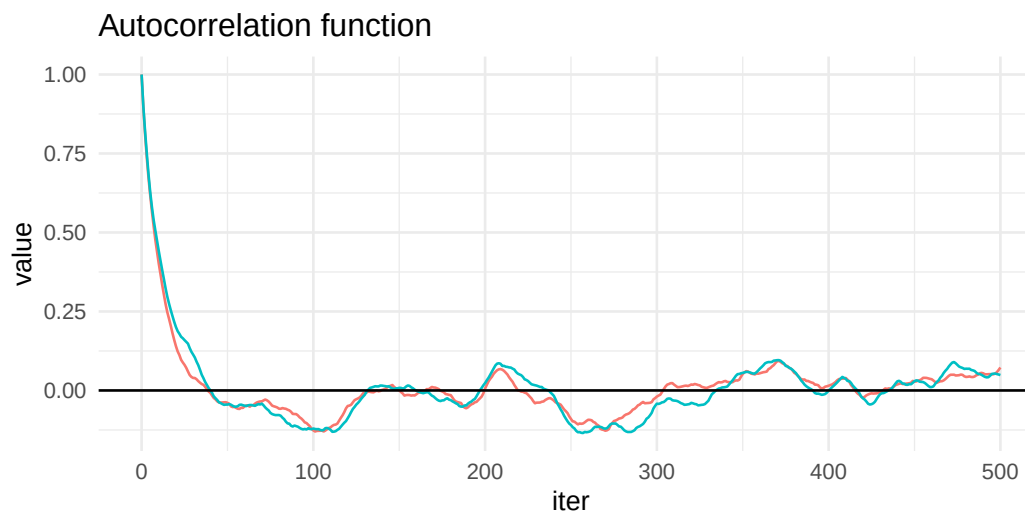
- This combines $\hat{R}$ and autocorrelation estimates
 - takes into account if the chains are not mixing (the chains have not converged)
- BDA3 has slightly different and less accurate equation.
- Compared to a method which computes the autocorrelation from a single chain, the multi-chain estimate has smaller variance

Time series analysis

- Estimation of τ

$$\tau = 1 + 2 \sum_{t=1}^{\infty} \hat{\rho}_t$$

where $\hat{\rho}_t$ is empirical autocorrelation

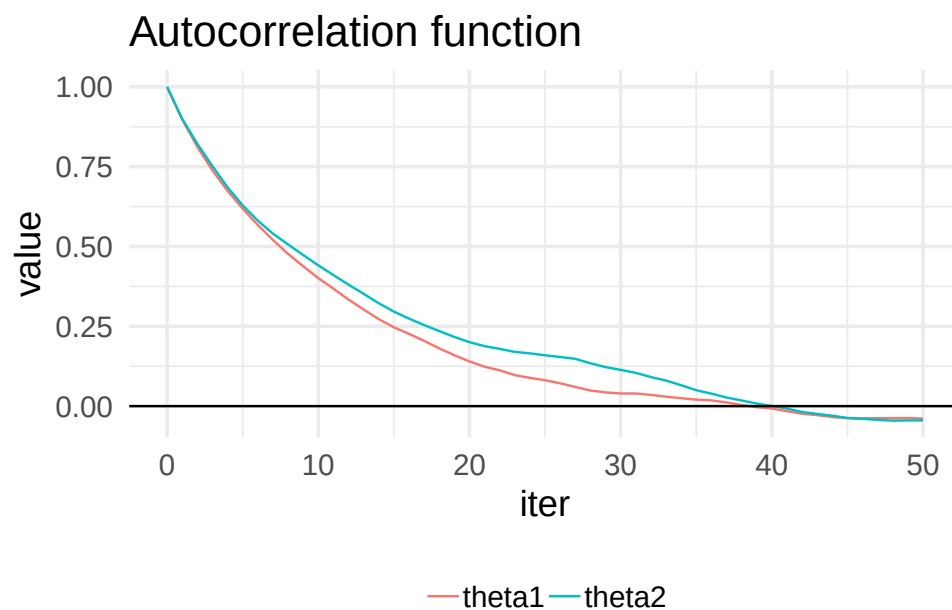


- $\hat{\rho}_t$ can be very noisy
 - noise is larger for longer lags (less observations)
 - less noisy estimate is obtained by truncating $\hat{\tau} = 1 + 2 \sum_{t=1}^T \hat{\rho}_t$

Geyer's adaptive window estimator

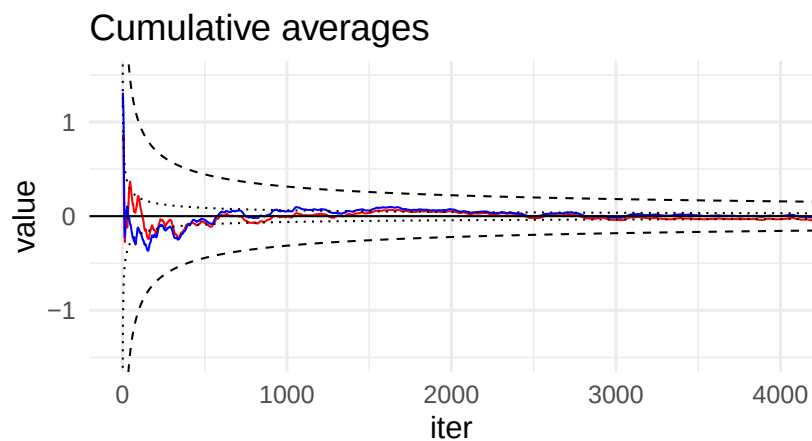
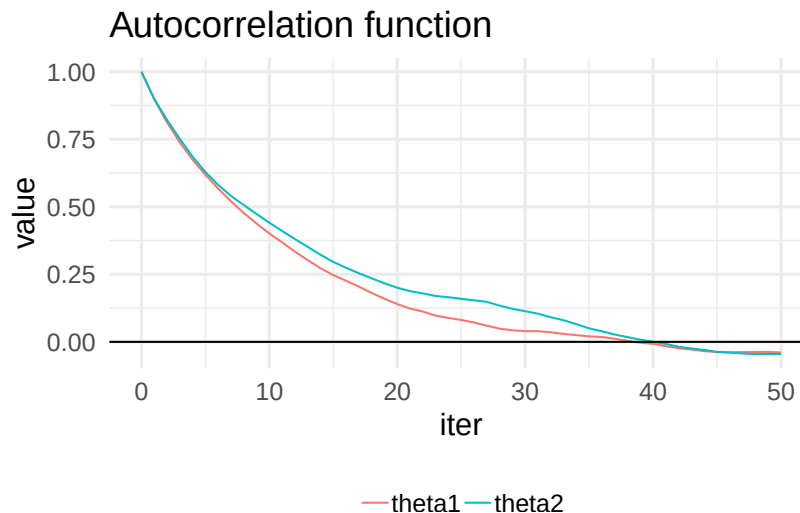
- Truncation can be decided adaptively
- Group the autocorrelations in pairs: $\Gamma_m = \rho_{2m} + \rho_{2m+1}$
- Theory guarantees that for MCMC chains these pair sums are never negative
 - if we estimate a negative $\hat{\Gamma}_m$, it can only be noise
- **Geyer's rule:** add up the pairs until the first negative one, then stop

$$\hat{\tau} = -1 + 2 \sum_{m=0}^M \hat{\Gamma}_m, \quad \text{ESS} \approx S / \hat{\tau}$$



Effective sample size

Effective sample size $ESS = S_{\text{eff}} \approx S/\hat{\tau}$

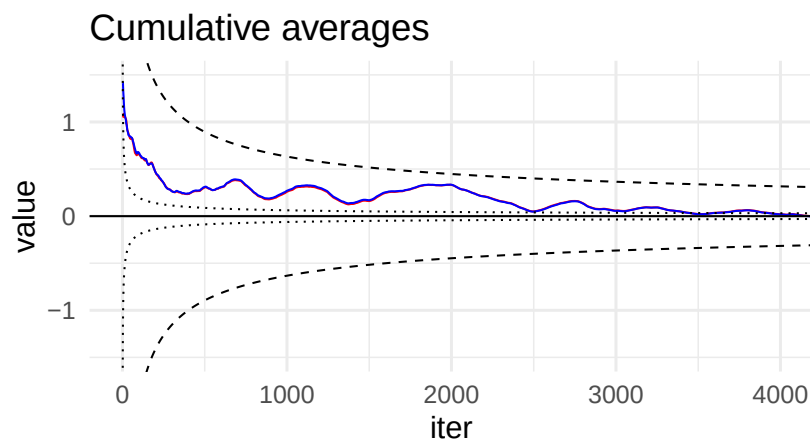
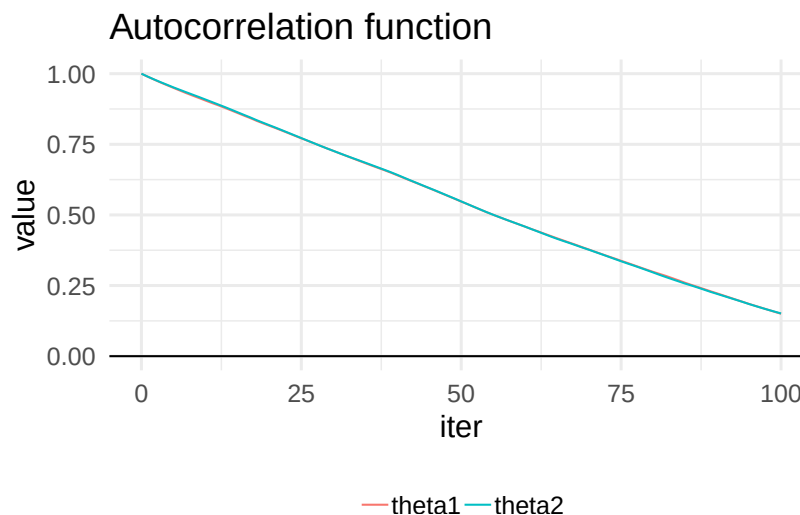
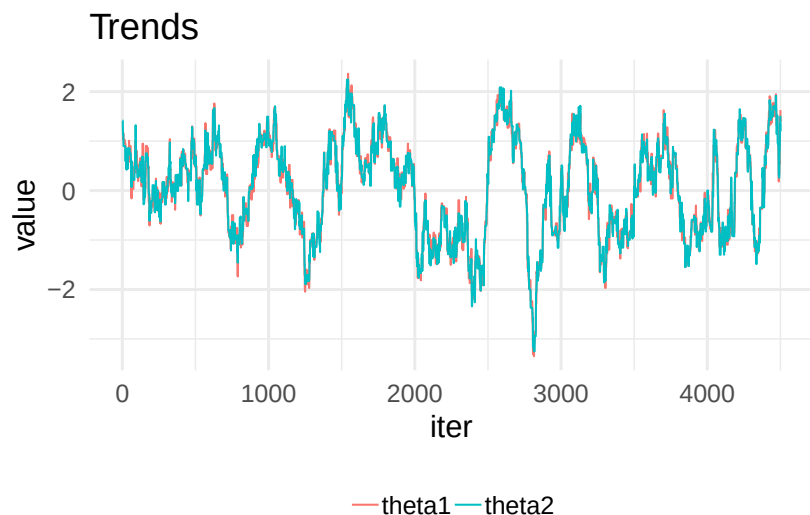


$$\hat{\tau} = 1 + 2 \sum_{t=1}^T \hat{\rho}_t$$
$$\approx 24$$

— theta1 — theta2 - - 95% interval for MCMC error 95% interval for indeper

Effective sample size

Effective sample size $ESS = S_{\text{eff}} \approx S/\hat{\tau}$

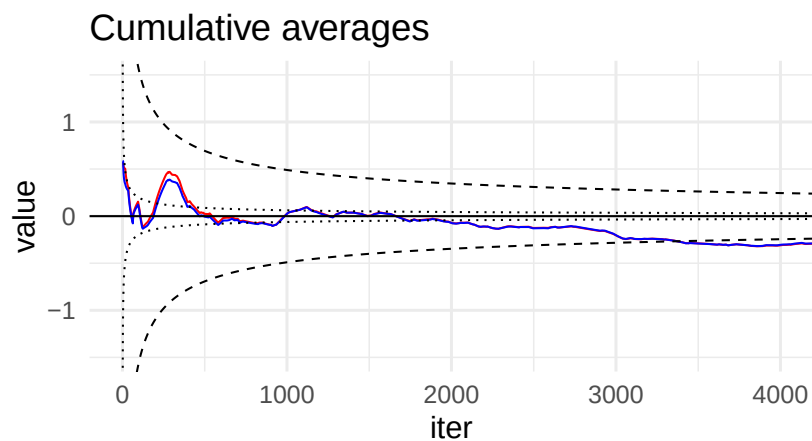
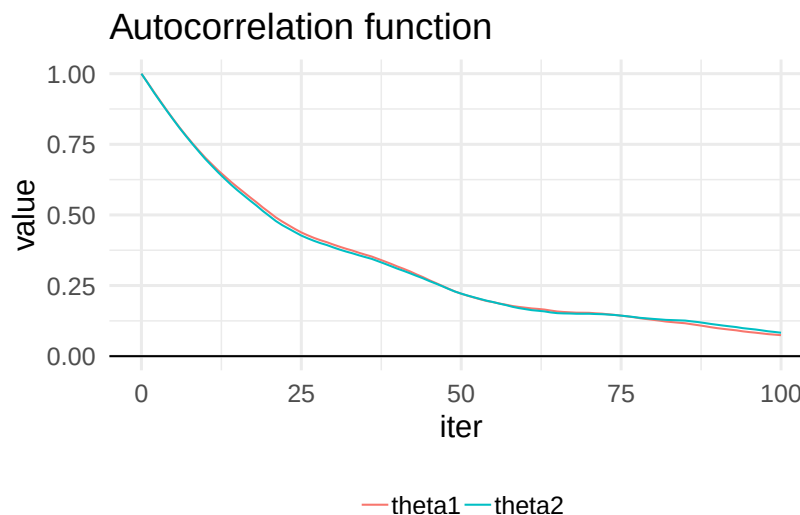
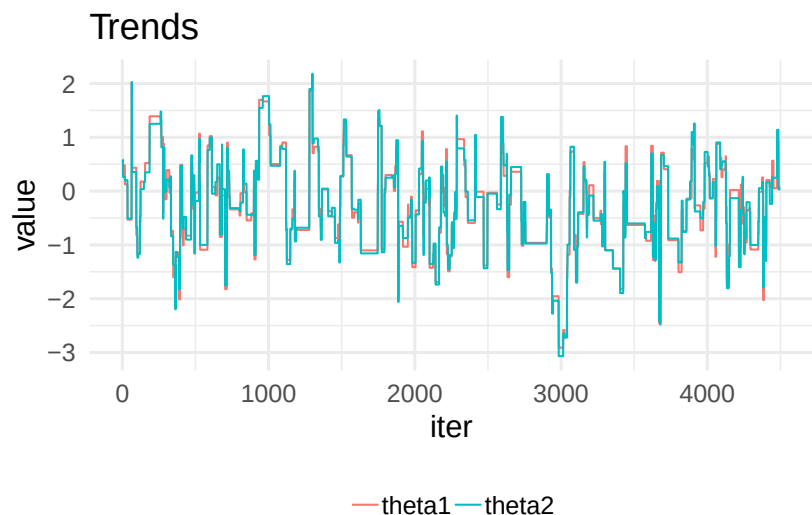


$$\hat{\tau} = 1 + 2 \sum_{t=1}^T \hat{\rho}_t$$
$$\approx 104$$

— theta1 — theta2 - - 95% interval for MCMC error 95% interval for indeper

Effective sample size

Effective sample size $ESS = S_{\text{eff}} \approx S/\hat{\tau}$



$$\hat{\tau} = 1 + 2 \sum_{t=1}^T \hat{\rho}_t$$
$$\approx 63$$

— theta1 — theta2 - - 95% interval for MCMC error 95% interval for indeper

ESS and MCSE

- ESS and MCSE depend on the quantity
 - Bulk-ESS and Tail-ESS are useful diagnostic summaries, but eventually need to look at the ESS / MCSE for the quantity of interest

Diagnostic tools

For this week's assignment:

- $\hat{R}$, ESS, MCSE
 - `library(posterior)`
 - `th |> summarise_draws(Rhat=basic_rhat, ESS=mean_ess)`
 - `th |> summarise_draws(mean, mean_mcse)`
 - `th |> summarize_draws(~quantile(.x, probs = c(0.05, 0.95)))`
 - see demo11_5 and Digits case study for the examples how to use these
- trace, autocorrelation, density, scatter plots in R
 - `library(bayesplot)`
 - `mcmc_trace(th), mcmc_acf(th), mcmc_areas(th), ...`
 - see demo11_5 for the examples for more bayesplot examples
- Python
 - see ArviZ package